

Polymorphic coverage types*

ZHE ZHOU

Purdue University, USA
(email: zhou956@purdue.edu)

ASHISH MISHRA

Indian Institute of Technology Hyderabad, India
(email: mishraashish@cse.iith.ac.in)

BENJAMIN DELAWARE

Purdue University, USA
(email: bendy@purdue.edu)

SURESH JAGANNATHAN

Purdue University, USA
(email: suresh@cs.purdue.edu)

Abstract

Test input generators are an important part of property-based testing (PBT) frameworks. Because PBT is intended to test deep semantic and structural properties of a program, the outputs produced by these generators can be complex data structures, constrained to satisfy properties the developer believes are most relevant to testing the function of interest. An important feature expected of these generators is that they be capable of producing *all* acceptable elements that satisfy the function's input type and generator-provided constraints. However, it is not readily apparent how we might validate whether a particular generator's output satisfies this *coverage* requirement. Typically, developers must rely on manual inspection and post-mortem analysis of test runs to determine if the generator is providing sufficient coverage; these approaches are error-prone and difficult to scale as generators become more complex. To address this important concern, we present a new refinement type-based verification procedure for validating the coverage provided by input test generators, based on a novel interpretation of types that embeds “*must-style*” underapproximate reasoning principles as a fundamental part of the type system. The types associated with expressions now capture the set of values *guaranteed* to be produced by the expression, rather than the typical formulation that uses types to represent the set of values an expression *may* produce. We have formalized the notion of *coverage types* in a rich core language that supports higher-order functions and inductive datatypes. To better support checking coverage properties of real-world test generators, we extend this type system with type and qualifier polymorphism. These extensions enable our system to statically verify coverage guarantees of test input generators constructed using the sort of monadic combinators found in most PBT frameworks. We have implemented a coverage type checker for OCaml programs based on this core calculus, and present a detailed evaluation of the utility of our ideas using a corpus of benchmarks drawn from both the PBT literature and open-source projects.

*Portions of this work previously appeared in the proceedings of the 2023 ACM Conference on Programming Language Design and Implementation [69].



© 2026 Copyright held by the owner/author(s).

This work is licensed under a Creative Commons Attribution 4.0 International License.

1 Introduction

Property-based testing (PBT) is a popular technique for automatically testing deep semantic and structural properties of programs. Originally pioneered by the QuickCheck [3] library for Haskell, PBT frameworks now exist for many programming languages, including QCheck [7], JavaScript [15], Rust [55], Python [27], Scala [56], and Rocq/Coq [36]. The PBT methodology rests on two key components: *executable properties* that capture the expected input-output behaviors of the program under test, and *test input generators* that generate random values of the input types needed to validate these behaviors. In contrast to unit tests, which rely on single examples of inputs and outputs, generators are meant to provide a *family* of inputs against which programs can be tested, with the goal of ensuring the set of generated tests provide good coverage of all possible inputs. In order to prune out irrelevant inputs, PBT frameworks allow users to define custom generators that reflect the specific shape of data that the developer believes is most likely to trigger interesting (aka faulty) behavior. As one simple example, to test a tree compression or balancing function, the developer may want to use a generator that produces n -ary trees with randomly chosen height and arity but whose leaves are ordered according to a user-provided ordering relation.

Given the critical role they play in the assurance case provided by PBT frameworks, it is reasonable to ask what constitutes a “good” specification for a test generator. For our example, one answer could be that it should only produce ordered trees. Of course, this is not a very satisfactory characterization of the behavior we desire: the “constant” generator that always produces trees of height one trivially meets this specification, but it is unlikely to produce useful tests for a compression function! Ideally, we would like a generator to intelligently enumerate the space of *all* possible ordered trees, thereby helping to maximize the likelihood of finding bugs in the function under test. Because defining such an enumeration procedure for arbitrary datatypes can be hard, even when complete enumeration is computationally feasible, PBT frameworks instead give developers the ability to assemble generators for complex data structures *compositionally*, building on generators for simpler types where randomly sampling elements of the type is straightforward and sufficient. For example, we could implement an ordered tree generator in terms of a primitive random number generator that is used to non-deterministically select the height, arity, and elements of a candidate tree, checking (or enforcing) the orderliness of the tree before returning it as a feasible test input. Although the random number generator might provide a guarantee that its underlying probability density function (PDF) is always non-zero on all elements in its sample space, determining that a tree generator that is built using it can actually enumerate all the ordered trees desired is a substantially harder problem. Even if we know the generator is capable of eventually yielding all trees, constraints imposed by the function’s precondition might require the generator to perform further filtering or transformations over generated trees. However, proving that any filtering operations the generator uses do not mistakenly prune out valid ordered trees or that any transformations the generator performs over candidate trees preserve the elements of the random tree being transformed, pose additional challenges. In other words, verifying that the generator is *complete* with respect to our desired orderliness property entails reasoning that is independent of the behavior of the primitive generators used to build the tree. Consequently, we require some alternative mechanism to help qualify the part of the target function’s input space the generator is actually guaranteed to cover. Devising such a mechanism is challenging precisely because the properties that need to be tested may

impose complex structural and semantic constraints on the generated output (e.g., requiring that an output tree be a binary search tree, or that it satisfies a red-black property, etc.); the complexity of these constraints is directly correlated to the sparseness of the function’s input space preconditions.

```

1 type 'a tree =
2   | Leaf
3   | Node of ('a * 'a tree * 'a tree)
4 let rec bst_gen (lo: int) (hi: int): int tree =
5   if lo + 1 >= hi then Leaf else
6   (* Leaf  $\oplus$  *)
7   (let (x: int) = int_range (lo + 1, hi - 1) in
8     Node (x, bst_gen lo x, bst_gen x hi))

```

Fig. 1. A BST generator. Failing to uncomment line 6 results in the generator never producing trees that contain only a subset of the elements in the interval between `lo` and `hi`, which is inconsistent with the developer’s intent.

To illustrate this distinction more concretely, consider the input test generator shown in Figure 1 that is intended to generate all binary search trees (BSTs) whose elements are in the interval `lo` to `hi`. If we ignore the comment on line 6, we can conclude this generator always produces a non-empty BST whenever `lo < hi`. While the generator is correct - it always generates a well-formed BST - it is also incomplete; the call `bst_gen 0 10`, for example, will never produce a tree containing just `Leaf` or a tree with a shape like `Node(1, Leaf, Leaf)`, even though these instances are valid trees consistent with the constraints imposed by the generator’s argument bounds. In fact, this implementation *never* generates a BST that only contains a proper subset of the elements that reside within the interval defined by `lo` and `hi`. By uncommenting line 6, however, we allow the generator to non-deterministically choose (via operator $\oplus$) to either return a `Leaf` or left and right BST subtrees based on the value returned by the `int_range` generator, enabling it to potentially produce BSTs containing all valid subsets of the provided interval, thus satisfying our desired completeness behavior. The subtleties involved in reasoning about such coverage properties are clearly non-trivial. We reiterate that recognizing the distinction between these two implementations is not merely a matter of providing a precise output type capturing the desired sortedness property of a BST: the incomplete implementation clearly satisfies such a type! Furthermore, simply knowing that the underlying `int_range` generator used in the implementation samples all elements within the range of the arguments it is provided is also insufficient to conclude that the BST generator can yield *all* possible BSTs within the supplied interval. Similar observations have led prior work to consider ways to improve a generator’s coverage through mechanisms such as fuzzing [10, 34], or to automatically generate complete-by-construction generators for certain classes of datatypes [35].

In contrast to these approaches, this paper embeds the notion of coverage as an integral part of a test input generator’s *type* specification. By doing so, a generator’s type now specifies the set of behaviors the generator is *guaranteed* to exhibit; a well-typed generator is thus guaranteed to produce *every possible* value satisfying a desired structural property, e.g., that the repaired (complete) version of `bst_gen` is capable of producing every valid BST. By framing the notion of coverage in type-theoretic terms, our approach neither requires instrumentation of the target program to assess the coverage effectiveness of a candidate

generator (as in Lampropoulos et al. [34]) nor does it depend on a specific compilation strategy for producing generators (as in Lampropoulos et al. [35]). Instead, our approach can automatically verify the coverage properties of an *arbitrary* test input generator, regardless of whether it was hand-written or automatically synthesized.

Key to our approach is a novel formulation of a *must*-style analysis [19, 20, 28] of a test input generator’s behavior in type-theoretic terms. In our proposed type system, we say an expression e has *coverage type* τ if every value contained in τ *must* be producible by e . Note how this definition differs from our usual notion of what a type represents: ordinarily, if e has type τ then we are allowed to conclude only that any value contained in τ *may* be produced by e . Informally, types interpreted in this usual way define an *overapproximation* of the values an expression e can yield, without obligating e to produce any specific such value. In contrast, coverage types define an *underapproximation* - they characterize the values an expression e has to produce, potentially eliding other values that e may also evaluate to. When the set of elements denoted by a generator’s (underapproximate) coverage type matches that of its (overapproximate) normal type, however, we can soundly assert that the generator is complete. As we illustrate in the remainder of the paper, this characterization allows us to reason about a program’s coverage behavior on the same formal footing as its safety properties.

In this sense, our solution can be seen as a type-theoretic interpretation of recently proposed Incorrectness Logics (IL) [37, 45, 53], in much the same way that refinement-type systems like Liquid Types [29, 62] relate to traditional program logics [25]. Despite the philosophical similarities with IL, however, we use underapproximate reasoning for a very different goal. While IL has been primarily used to precisely capture the conditions that will lead a program to fault, this work explores how type-based underapproximate reasoning can be used to verify the completeness properties of a test generator in the context of PBT.

This interpretation leads to a fundamental recasting of how types relate to one another: ordinarily, we are always allowed to assert that $\tau <: \top$. This means that any typing context that admits an expression with type τ can also admit that expression at a type with a logically weaker structure. In contrast, the subtyping relation for coverage types inverts this relation, so that $\top <: \tau$. Intuitively, $\top$ represents the coverage type that obligates an expression ascribed this type to be capable of producing *all* elements in τ . But, any context that requires an expression to produce all such elements can always guarantee that the expression will also produce a subset of these elements. In other words, we are always allowed to weaken an overapproximation (i.e., grow the set of values an expression may evaluate to), and strengthen an underapproximation (i.e., shrink the set of values an expression must evaluate to). Thus, in our setting, a random number generator over the integers has coverage type $\top$ under the mild assumption that its underlying PDF provides a non-zero likelihood of returning every integer. In contrast, a faulty computation like $1 \div 0$ has coverage type $\perp$ since there are no guarantees provided by the computation on the value(s) it must return. Here, $\perp$ represents a type that defines a degenerate underapproximation, imposing no constraints on the values an expression ascribed this type must produce.

In practice, developers often implement input generators using *test generator combinators* provided by the PBT framework. In this setting, generators are defined in terms of a monad that facilitates the compositional construction of complex generators from simpler ones in a modular way. The OCaml PBT framework QCheck [7], for example, ascribes the type `'a Gen.t` to such generators, as can be seen in the signature for the binary tree generator

```

1  val return: 'a -> 'a Gen.t
2  val bind: 'a -> ('a -> 'b Gen.t) -> 'b Gen.t
3  val frequency: (int -> (int * 'a Gen.t)) -> 'a Gen.t
4  val fix: (int -> (int -> 'a -> 'a Gen.t) -> 'a Gen.t) -> int -> 'a Gen.t
5
6  type 'a tree = Leaf of 'a | Node of 'a tree * 'a tree
7  let tree_gen : int -> int tree Gen.t = fix
8    (fun n self -> match n with
9      | 0 -> return err
10     | 1 -> bind nat_gen (fun x -> return (Leaf x))
11     | n ->
12       frequency
13         (function
14           | 0 -> (1, bind nat_gen (fun x -> return (Leaf x)))
15           | _ -> (2, bind (self n/2) (fun tr1 ->
16                         bind (self n/2) (fun tr2 ->
17                           return (Node (tr1, tr2))))))
18   )

```

Fig. 2. A tree generator expressed using QCheck.

`tree_gen` shown in Fig. 2. This generator is implemented by 4 combinators: in addition to the basic monad operators (`return` and `bind`), `tree_gen` also uses the `frequency` combinator to sample from multiple generators according to the given weights, as well as the `fix` combinator to recursively construct trees of greater height from smaller ones. Extending coverage types to support monadic combinators and type polymorphism (on both types and qualifiers) requires significant extensions over the core monomorphic, non-monadic formulation.

In summary, this article makes the following contributions:

- (1) Introduces the notion of *coverage types*, types that characterize the values that an input test generator is guaranteed to produce.
- (2) Formalizes the semantics of coverage types in an ML-like functional language with support for higher-order functions and inductive datatypes.¹
- (3) Develops a bi-directional type-checking algorithm for coverage types in this language.
- (4) Extends this core calculus and typing algorithm to support coverage type polymorphism and monadic composition.
- (5) Incorporates these ideas in a tool (Poirot) that operates over OCaml programs equipped with input generators and typed using coverage types.
- (6) Presents an extensive empirical evaluation justifying the utility of coverage types, by using Poirot to verify the coverage properties of both hand-written generators drawn from real-world open-source PBT applications and automatically synthesized generators, covering a rich class of datatypes and their structural properties.

The remainder of this article is structured as follows. In the next section, we present an informal overview of the key features of our type system. Section 3 presents the syntax and semantics for a core call-by-value higher-order functional language with inductive datatypes that we use to formalize our approach. Section 4 describes our coverage type system, and its

¹A Rocq formalization of this calculus, its type system, and its metatheory (i.e., Theorem 5.3) is provided on Zenodo [70].

Table 1. Examples of overapproximate and underapproximate (coverage) typings. We use $\vdash$ and $\nvdash$ to identify whether a term can or cannot be assigned the corresponding type, resp. The constant `err` represents a special error value, which causes the program to halt when encountered.

<code>int_gen()</code>	$\vdash [v: \text{int} \mid \top]$ $\vdash \{v: \text{int} \mid \top\}$	$\vdash [v: \text{int} \mid v = 1 \vee 2]$ $\nvdash \{v: \text{int} \mid v = 1 \vee 2\}$	$\vdash [v: \text{int} \mid v = 1]$ $\nvdash \{v: \text{int} \mid v = 1\}$	$\vdash [v: \text{int} \mid \perp]$ $\nvdash \{v: \text{int} \mid \perp\}$
<code>1</code>	$\vdash [v: \text{int} \mid v = 1]$ $\vdash \{v: \text{int} \mid v = 1\}$	$\vdash [v: \text{int} \mid \perp]$ $\nvdash \{v: \text{int} \mid \top\}$	$\vdash \{v: \text{int} \mid \top\}$ $\nvdash [v: \text{int} \mid v = 1 \vee 2]$	$\vdash [v: \text{int} \mid v = 1 \vee 2]$ $\nvdash \{v: \text{int} \mid \perp\}$
<code>err</code>	$\vdash [v: \text{int} \mid \perp]$ $\nvdash \{v: \text{int} \mid \top\}$	$\nvdash [v: \text{int} \mid \top]$ $\nvdash \{v: \text{int} \mid v = 1 \vee 2\}$	$\nvdash [v: \text{int} \mid v = 1 \vee 2]$ $\nvdash \{v: \text{int} \mid v = 1\}$	$\nvdash [v: \text{int} \mid v = 1]$ $\nvdash \{v: \text{int} \mid \perp\}$

metatheory is given in Section 5. A bidirectional typing algorithm is then given in Section 6. We describe details about the implementation of Poirot and provide benchmark results in Section 9. Related work and conclusions are given in Sections 10 and 11.

A preliminary version of this article appeared at PLDI 2023 [69]. We extend that version with additional details and examples. The bulk of the new material is a polymorphic extension to the original coverage type system (Sec. 7). We use this extension to define a coverage monad and to type-check generator combinators inspired by OCaml’s QCheck PBT framework (Sec. 8). We also substantially extend the evaluation with a new case study that uses these features to verify coverage properties of real-world input generators taken from open-source PBT projects (Sec. 9).

2 Overview

Before presenting the full details of our type system, we begin with an informal overview of its key features.

Base types. In the following, we write $[v: b \mid \phi]$ to denote the coverage type that qualifies the base type b using the predicate ϕ . As described in the previous section, an application of the primitive built-in generator for random numbers: `int_gen` : `unit` $\rightarrow$ `int` has the coverage type `int_gen ()` : $[v: \text{int} \mid \top]$. We use brackets [...] to emphasize that a coverage type has a different meaning from the types typically found in other refinement type systems [29, 62] where a qualified type b , written as $\{v: b \mid \phi\}$, uses a predicate ϕ to constrain the set of values a program *might* evaluate to. To illustrate this distinction, consider the combinations of expressions and types shown in Table 1. These examples demonstrate the previous observation that it is always possible to strengthen the refinement predicate used in an underapproximate type and weaken such a predicate in an overapproximate type. A similar phenomenon appears in IL’s rule of consequence, which inverts the direction of the implications on pre- and postconditions in the overapproximate version of the rule. As a result, the bottom type $[v: \text{int} \mid \perp]$ is the universal supertype in our type hierarchy, as it places no restrictions on the values a term *must* produce. Thus, we sometimes abbreviate $[v: \text{int} \mid \perp]$ as `int`, since the information provided by both types is the same. Importantly, the coverage type for the error term (`err`) can *only* be qualified with $\perp$, since an erroneous computation is unconstrained with respect to the values it is obligated to produce.

Coverage types can also qualify inductive datatypes, like lists and trees. In particular, the complete generator for BSTs presented in the introduction can be successfully type-checked

```

let even_gen () =
  let (n: int) = int_gen () in
  let (b: bool) = n mod 2 == 0 in
  if b then n else err

```

Fig. 3. An even number generator defined in terms of an integer number generator.

using the following result type:

$$[v: \text{int tree} \mid bst(v) \wedge \forall u, mem(v, u) \implies lo < u < hi]$$

where $bst(v)$ and $mem(v, u)$ are *uninterpreted predicates* used to encode semantic properties of the datatype. In the type given above, the qualifier requires that bst_gen 's result is a BST (encoded by the predicate $bst(v)$) and that every element u stored in the tree (encoded by the predicate $mem(v, u)$) is between lo and hi ; the coverage type thus constrains the implementation to produce *all* trees that satisfy this qualifier predicate. In contrast, the incomplete version of the generator (i.e., the implementation that does not allow prematurely terminating tree generation with a `Leaf` node) could only be type-checked using the following (stronger) type:

$$[v: \text{int tree} \mid bst(v) \wedge \forall u, mem(v, u) \iff lo < u < hi]$$

This signature asserts that all trees produced by the generator are BSTs, that any element contained in the tree is within the interval bounded by lo and hi , *and* moreover, any element in that interval *must* be included in the tree. The subtle difference between the two implementations, reflected in the different implication constraints expressed in their respective refinements, precisely captures how their coverage properties differ.

Control flow. Just as underapproximate coverage types invert the standard overapproximate subtyping relationship, they also invert the standard relationship between a control flow construct and its subexpressions. To see how, consider the simple generator for even numbers shown in Fig. 3. When the integer generator, `int_gen()`, yields an odd number, `even_gen` faults; otherwise it simply returns the generated number. Consider the following type judgment that arises when type checking this program:

$$\begin{aligned}
 &n: [v: \text{int} \mid \top], b: [v: \text{bool} \mid v \iff n \bmod 2 = 0] \\
 &\vdash \text{if } b \text{ then } n \text{ else err} : [v: \text{int} \mid v \bmod 2 = 0]
 \end{aligned} \tag{1}$$

Intuitively, this judgment asserts that the `if` expression covers all even numbers (i.e., has the type $[v: \text{int} \mid v \bmod 2 = 0]$) assuming that the *local variable* n can be instantiated with an arbitrary number, and that the variable b is true precisely when n is even. Notice how the typing context encodes the potential control-flow path that *must* reach the non-faulting branch of the conditional expression. Enforcing the requirement that the conditional be able to return all even numbers does *not* require each of its branches to be a subtype of the expected type, in contrast to standard type systems. Our type system must instead establish that, *in total*, the values produced by each of the branches cover the even numbers. Because the false branch of the conditional faults, it is only typeable at the universal supertype, i.e., $[v: \text{int} \mid \perp]$. Thus, if the standard subtyping relationship between this conditional and its branches held, it could only be typed at $[v: \text{int} \mid \perp]$! This is not the case in our setting, as the `true` branch contributes all the desired outputs. Formally, this property is checked by the

following assumption of the coverage typing rule for conditionals:

$$\begin{aligned} n:[v: \text{int} \mid \top], b:[v: \text{bool} \mid v \iff n \bmod 2 = 0] \\ \vdash [v: \text{int} \mid (b \wedge v = n) \vee (\neg b \wedge \perp)] <: [v: \text{int} \mid v \bmod 2 == 0] \end{aligned}$$

The $b \wedge v = n$ and $\neg b \wedge \perp$ subformulas correspond to the types of the true and false branches², respectively. Taking the disjunction of these two formulas describes the set of values that can be produced by either branch³; this subtyping relationship guarantees this type is at least as large as the type expected by the entire conditional. To check that this subtyping relationship holds, our type checker generates the following formula:

$$\forall v, (v \bmod 2 = 0) \implies (\exists n, \top \wedge \exists b, b \iff n \bmod 2 = 0 \wedge (b \wedge v = n) \vee (\neg b \wedge \perp)) \quad (2)$$

This formula aligns with the intuitive meaning of (1): in our type system, coverage types of variables in the typing context tell us what values they must (at least) produce. When checking whether a particular subtyping or typing relationship holds, we are free to choose *any* instantiation of the variables that entails the desired property. Accordingly, in (2), the variables n and b are *existentially* quantified to indicate there exists an execution path that instantiates these local variables in a way that produces the output v , instead of being universally quantified as they would be in a standard refinement type system.

Function types. To type functions, most refinement type systems add a restricted form of the *dependent function types* found in full-spectrum dependent type systems. Such types allow the qualifiers in the result type of a function to refer to its parameters, enabling the expression of rich safety conditions governing the arguments that may be supplied to the function. To see how this capability might be useful in our setting, consider the test generator `bst_gen` from the introduction. The complete version of this function produces all BSTs whose elements fall between the range specified by its two parameters, `lo` and `hi`. For the bounds `0` and `3`, the application `bst_gen 0 3` can be typed as: $[v: \text{int} \mid \text{bst}(v) \wedge \forall u, \text{mem}(v, u) \implies 0 < u < 3]$. Using the standard typing rule for functions, the only way to encode this relationship in the type of `bst_gen` is:

$$\begin{aligned} \text{lo}: [v: \text{int} \mid v = 0] \rightarrow \text{hi}: [v: \text{int} \mid v = 3] \rightarrow \\ [v: \text{int tree} \mid \text{bst}(v) \wedge \forall u, \text{mem}(v, u) \implies 0 < u < 3] \end{aligned}$$

Of course, this specification fails to account for the behaviors of `bst_gen` when supplied with different bounds: for example, the application `bst_gen 2 7` will fail to typecheck against this type.

Since the desired coverage property of `bst_gen` fundamentally depends on the kinds of inputs given to it, our type system includes dependent products of the form:

$$\begin{aligned} \text{lo}: \{v: \text{int} \mid \top\} \rightarrow \text{hi}: \{v: \text{int} \mid \text{lo} \leq v\} \rightarrow \\ [v: \text{int tree} \mid \text{bst}(v) \wedge \forall u, \text{mem}(v, u) \implies \text{lo} < u < \text{hi}] \end{aligned}$$

We use the notation $\{\dots\}$ to emphasize that the argument types of a dependent arrow have a similar purpose and interpretation as in standard refinement type systems. Thus, the above

²As is standard in dependent type systems, the types of both branches have been refined to reflect the path conditions under which they will be executed.

³This is similar to how the derived rule of choice in IL uses disjunction to reason about both branches of a nondeterministic choice statement.


```

let bst_gen_low_bound (low: int) =
  let (high: int) = int_gen () in
  bst_gen low high

```

Fig. 4. This function generates a BST with a supplied lower bound, low.

type can be read as “if the inputs `lo` and `hi` are *any* number such that $lo \leq hi$, then the output *must* cover all possible BSTs whose elements are between `lo` and `hi`”. Using this type for `bst_gen` allows our system to seamlessly type-check both `(bst_gen 0 3)` and `(bst_gen 2 7)`. Our typing algorithm will furthermore flag the call `(bst_gen 3 1)` as being ill-typed, since the function’s type dictates that the generator’s second argument (1) *may* only be greater than or equal to its first (3).

Function application. Since the type of a function parameter is interpreted as a normal (overapproximate, “may”) refinement type, while arguments in an application may be typed using (underapproximate, “must”) coverage types, we need to be able to bridge the gap between may and must types when typing function applications. Intuitively, our type system does so by ensuring that the set of values in the coverage type of the argument has a nonempty overlap with the set of possible values expected by the function. We establish this connection by using the fact that the typing context captures the control flow paths that may and must exist when the function is called. To illustrate this intuition concretely, consider the function `bst_gen_low_bound` shown in Fig. 4. This function generates all non-empty BSTs whose elements are integers with the lower bound given by its parameter. The judgment we need to check is of the form:

$$\begin{aligned}
 & \text{bst_gen} : \text{lo}:\{v: \text{int} \mid \top\} \rightarrow \text{hi}:\{v: \text{int} \mid \text{lo} \leq v\} \rightarrow [v: \text{int tree} \mid \dots], \\
 & \text{low}:\{v: \text{int} \mid \top\}, \text{high}:[v: \text{int} \mid \top] \\
 & \vdash \text{bst_gen low high} : \dots
 \end{aligned}$$

Note that the type for `low` is a normal refinement type that specifies a safety condition for function `bst_gen_low_bound`, namely that `low` *may* be any number. In contrast, the type for `high` is a coverage type, representing the result of `int_gen()` that indicates that it *must* (i.e., guaranteed to) be any possible integer. However, the signature for `bst_gen` demands that parameter `hi` only be supplied values greater than its first argument (`lo`); we incorporate this requirement by *strengthening* `high`’s type (via a subsumption rule) to reflect this additional constraint when typing the body of the `let` expression in which `high` is bound. This strengthening, which is tantamount to a more refined underapproximation, allows us to typecheck the application `(bst_gen low high)` in the following context:

$$\begin{aligned}
 & \text{bst_gen} : \text{lo}:\{v: \text{int} \mid \top\} \rightarrow \text{hi}:\{v: \text{int} \mid \text{lo} \leq v\} \rightarrow [v: \text{int tree} \mid \dots], \\
 & \text{low}:\{v: \text{int} \mid \top\}, \text{high}:[v: \text{int} \mid \text{low} \leq v] \\
 & \vdash \text{bst_gen low high} : \dots
 \end{aligned}$$

The coverage type associated with `high` guarantees that `int_gen()` must produce values greater than `low` (along with possibly other values). To ensure that the result type of the call reflects the underapproximate (coverage) dependences that exist between `low` and `high`, we

Variables	$x, y, z, f, u, \dots$
Data constructors	$d ::= () \mid \text{true} \mid \text{false} \mid 0 \mid \text{S} \mid \text{Cons} \mid \text{Nil} \mid \text{Leaf} \mid \text{Node}$
Constants	$c ::= \mathbb{B} \mid \mathbb{N} \mid \mathbb{Z} \mid \dots \mid d \bar{c}$
Operators	$op ::= d \mid + \mid == \mid < \mid \text{mod} \mid \text{nat_gen} \mid \text{int_gen} \mid \dots$
Values	$v ::= c \mid op \mid x \mid \text{fun } (x:\tau) \rightarrow e \mid \text{fix } (f:\tau) \text{ fun } (x:\tau) \rightarrow e$
Terms	$e ::= v \mid \text{err} \mid \text{let } x = e \text{ in } e \mid \text{let } x = op \bar{v} \text{ in } e$ $\mid \text{let } x = v \text{ v in } e \mid \text{match } v \text{ with } \overline{d \bar{y}} \rightarrow e$
Base Types	$b ::= \text{unit} \mid \text{bool} \mid \text{nat} \mid \text{int} \mid \text{float} \mid \text{char} \mid \text{string}$ $\mid b \times b \mid \{x:b\} \mid b \text{ option} \mid b \text{ list} \mid b \text{ tree} \mid \dots$
Basic Types	$t ::= b \mid t \rightarrow t$
Uninterpreted Functions	$up ::= \text{emp} \mid \text{hd} \mid \text{mem} \mid \dots$
Literals	$l ::= c \mid x$
Propositions	$\phi ::= l \mid \perp \mid \top \mid op(\bar{l}) \mid up(\bar{x})$ $\mid \neg\phi \mid \phi \wedge \phi \mid \phi \vee \phi \mid \phi \implies \phi \mid \forall u:b. \phi \mid \exists u:b. \phi$
Refinement Types	$\tau ::= [v: b \mid \phi] \mid \{v: b \mid \phi\} \mid x:\tau \rightarrow \tau$
Type Contexts	$\Gamma ::= \emptyset \mid \Gamma, x:\tau$

Fig. 5. λ^U syntax.

introduce existential quantifiers in the type's qualifier:

```
..., low: {v: int |  $\top$ }
 $\vdash [v: \text{int tree} \mid bst(v) \wedge \exists \text{high}, \text{low} \leq \text{high} \wedge \forall u, \text{mem}(v, u) \implies \text{low} < u < \text{high}]$ 
```

This type properly captures the behavior of the generator: it is guaranteed to generate all BSTs characterized by a lower bound given `low` such that there exists an upper bound `high` where $\text{low} \leq \text{high}$ and in which every element in the tree is contained within these bounds.

Summary. Coverage types invert many of the expected relationships that are found in a normal refinement type system. Here, qualifiers provide an *underapproximation* of the values that an expression may evaluate to, in contrast to the typically provided *overapproximation*. This, in turn, causes the subtyping relation to invert the standard relationship entailed by logical implication between type qualifiers. Our coverage analysis also considers the *disjunction* of the coverage guarantees provided by the branches of control-flow constructs, instead of their *conjunction*. Finally, when applying a function with a dependent arrow type to a coverage type, we check semantic inclusion between the overapproximate and underapproximate constraints provided by the two types, and manifest the paths that witness the elements guaranteed to be produced by the coverage type through existentially-quantified variables in the application's result type.

3 Language

Terms. In order to formalize our typed-based verification approach of input test generators, we introduce a core calculus for test generators, λ^U . The language, whose syntax is summarized

in Fig. 5, is a call-by-value lambda-calculus with pattern-matching, inductive datatypes, and well-founded (i.e., terminating) recursive functions whose argument must be structurally decreasing in all recursive calls made in the function’s body. The syntax of λ^U is expressed in monadic normal-form (MNF) [24], a variant of A-Normal Form (ANF) [17] that allows nested let-bindings. Terms in a standard lambda-calculus can be expressed as syntax sugar in ANF, e.g., an application ($e_1 e_2$) can be defined as:

$$e_1 e_2 \doteq \text{let } f = e_1 \text{ in let } x = e_2 \text{ in let } y = f x \text{ in } y$$

To enable type-checking, the parameters of lambda functions (i.e. **fun** $(x:\tau) \rightarrow e$) and fixpoint functions (i.e. **fix** $(f:\tau)$ **fun** $(x:\tau) \rightarrow e$) are also annotated with coverage types. The language additionally allows faulty programs to be expressed using the error term **err**. As discussed in Sec. 2, this term is important in our investigation because coverage types capture an expression’s reachability properties, and we need to ensure the guarantees offered by such types are robust even in the presence of stuck computations induced by statements like **err**. The language is also equipped with primitive operators to generate natural numbers, integers, etc. (**nat_gen** $()$, **int_gen** $()$, ...) that can be used to express various kinds of non-deterministic behavior relevant to test input generation. As an example, the $\oplus$ choice operator used in Fig. 1 can be defined as:

$$e_1 \oplus e_2 \doteq \text{let } n = \text{int_gen } () \text{ mod } 2 \text{ in match } n \text{ with } 0 \rightarrow e_1 \mid _ \rightarrow e_2$$

Note that the primitive generators of λ^U are completely agnostic to the specific sampling strategy they employ, as long as they ensure every value in their range has a nonzero likelihood of being generated. λ^U has a completely standard small-step operational semantics.

Types. Like other refinement type systems [29], λ^U supports three classes of types: *base types*, *basic types*, and *refinement types*. Base types (b) include primitive types such as **unit**, **bool**, **nat**, etc., product types ($b \times b$), record types $\{\overline{x:b}\}$, and inductive datatypes (e.g., **int list**, **bool tree**, **int list list**, etc.). Basic types (t) extend base types with function types. Refinement types (τ) qualify base types with both underapproximate and overapproximate propositions, expressed as predicates defined in first-order logic (FOL). Function parameters can also be qualified with overapproximate refinements that specify when it is safe to apply this function. In contrast, the return type of a function can only be qualified using an underapproximate refinement, reflecting the coverage property of the function’s result and thus characterizing the values the function is guaranteed to produce. The type context is defined normally, i.e., as a sequence of variable-type bindings consisting of overapproximate refinement types, underapproximate coverage types, and arrow (function) types.

Qualifiers. To express rich shape properties over inductive datatypes, we allow propositions to reference uninterpreted predicates, as it is straightforward to generate verification conditions using these uninterpreted predicates that can be handled by an off-the-shelf theorem prover like Z3 [8]. As we describe in Section 6, our type-checking algorithm imposes additional constraints on the form propositions can take, in order to ensure that its validity is decidable. In particular, we require that uninterpreted predicates in our language are stratified and that the Z3 queries generated by our type checker to check refinement validity are always over

Type Erasure

$$\boxed{[\tau] \quad [\Gamma]}$$

$$[\{v: b \mid \phi\}] \doteq b \quad [\llbracket v: b \mid \phi \rrbracket] \doteq b \quad [x:t \rightarrow \tau] \doteq [t] \rightarrow [\tau] \quad [\emptyset] \doteq \emptyset \quad [x:\tau, \Gamma] \doteq x:[\tau], [\Gamma]$$

Well-Formedness

$$\boxed{\vdash^{\text{WF}} \Gamma \quad \Gamma \vdash^{\text{WF}} \tau}$$

$$\begin{array}{c} \frac{\vdash^{\text{WF}} \Gamma \quad [\Gamma], v:b \vdash_t \phi : \text{bool}}{\Gamma \vdash^{\text{WF}} \llbracket v: b \mid \phi \rrbracket} \text{WFBASE} \quad \frac{\Gamma, x:\{v: b \mid \phi\} \vdash^{\text{WF}} \tau}{\Gamma \vdash^{\text{WF}} x:\{v: b \mid \phi\} \rightarrow \tau} \text{WFARG} \\ \\ \frac{\Gamma, x:(y:\tau_y \rightarrow \tau) \vdash^{\text{WF}} \tau}{\Gamma \vdash^{\text{WF}} x:(y:\tau_y \rightarrow \tau) \rightarrow \tau} \text{WFBRES} \\ \\ \frac{}{\vdash^{\text{WF}} \emptyset} \text{WFCTXEMP} \quad \frac{\vdash^{\text{WF}} \Gamma \quad \Gamma \vdash^{\text{WF}} \llbracket v: b \mid \phi \rrbracket}{\vdash^{\text{WF}} \Gamma, x:\{v: b \mid \phi\}} \text{WFCTXOBASE} \\ \\ \frac{\vdash^{\text{WF}} \Gamma \quad \Gamma \vdash^{\text{WF}} \llbracket v: b \mid \phi \rrbracket \quad \text{err} \notin \llbracket \llbracket v: b \mid \phi \rrbracket \rrbracket_{\Gamma}}{\vdash^{\text{WF}} \Gamma, x:\llbracket v: b \mid \phi \rrbracket} \text{WFCTXUBASE} \quad \frac{\vdash^{\text{WF}} \Gamma \quad \Gamma \vdash^{\text{WF}} y:\tau_y \rightarrow \tau}{\vdash^{\text{WF}} \Gamma, x:(y:\tau_y \rightarrow \tau)} \text{WFCTXARR} \end{array}$$

Subtyping

$$\boxed{\Gamma \vdash \tau_1 <: \tau_2}$$

$$\begin{array}{c} \frac{\llbracket \llbracket v: b \mid \phi_1 \rrbracket \rrbracket_{\Gamma} \subseteq \llbracket \llbracket v: b \mid \phi_2 \rrbracket \rrbracket_{\Gamma}}{\Gamma \vdash \llbracket v: b \mid \phi_1 \rrbracket <: \llbracket v: b \mid \phi_2 \rrbracket} \text{SUBUBASE} \quad \frac{\llbracket \{v:b \mid \phi_1\} \rrbracket_{\Gamma} \subseteq \llbracket \{v:b \mid \phi_2\} \rrbracket_{\Gamma}}{\Gamma \vdash \{v:b \mid \phi_1\} <: \{v:b \mid \phi_2\}} \text{SUBOBASE} \\ \\ \frac{\Gamma \vdash \tau_{21} <: \tau_{11} \quad \Gamma, x:\tau_{21} \vdash \tau_{12} <: \tau_{22}}{\Gamma \vdash x:\tau_{11} \rightarrow \tau_{12} <: x:\tau_{21} \rightarrow \tau_{22}} \text{SUBARR} \end{array}$$

Disjunction

$$\boxed{\Gamma \vdash \tau_1 \vee \tau_2 = \tau_3}$$

$$\frac{\llbracket \tau_1 \rrbracket_{\Gamma} \cap \llbracket \tau_2 \rrbracket_{\Gamma} = \llbracket \tau_3 \rrbracket_{\Gamma}}{\Gamma \vdash \tau_1 \vee \tau_2 = \tau_3} \text{DISJUNCTION}$$

Fig. 6. Auxiliary typing relations

effectively propositional (EPR) sentences (i.e., prenex-quantified formulae of the form $\exists^* \forall^* \varphi$ where φ is quantifier-free).

4 Type system

Similar to other contemporary refinement type systems [29], our type system assumes all terms are well-typed in a basic (aka non-refined) type system⁴, i.e., $\emptyset \vdash_t e : t$. For example, a term e that has refinement type τ has basic type $[\tau]$ where $[\dots]$ is a type erasure function, as shown in Fig. 6, which erases all qualifiers from refinement types and type contexts.

Despite superficial similarities to other contemporary type systems, the typing rules of λ^U differ in significant ways from those of its peers, due to the fundamental semantic distinction that arises when viewing types as an underapproximation and not overapproximation of program behavior.

Our type system depends on three auxiliary relations shown in Fig. 6. The first group defines well-formedness conditions on type contexts ($\vdash^{\text{WF}} \Gamma$) as well as refinement types under a particular type context ($\Gamma \vdash^{\text{WF}} \tau$). A type τ that is well-formed under a well-formed

⁴The basic typing rules, proofs of theorems, and the details of our evaluation are provided in technical report [71].

type context Γ needs to meet three criteria: (1) the qualifier in τ needs to be well-typed under the erasure of the current typing context in the basic type system (e.g., requirement $[\Gamma], v:b \vdash_{\tau} \phi : \text{bool}$ in WF_{BASE}), (2) overapproximate types may only appear in the domain of a function type (WF_{ARG}); and, (3) underapproximate coverage types may only appear in the range of a function type (only underapproximate and function type are allowed as parameter type in WF_{ARG} and WF_{RES} respectively). A type context is well-formed when all types in a type context are well-formed under the context derived from the former's bindings, moreover, the denotations⁵ of coverage base types ($[v: b \mid \phi]$) should not include err ($\text{WF}_{\text{CTXUBASE}}$). To understand the motivation for this constraint, observe that a type context in our setting provides a witness to feasible execution paths in the form of bindings to local variables. Accordingly, the type context $x:[v: \text{nat} \mid \perp]$ or $x:[v: \text{nat} \mid v > 0]$, $y:[v: \text{nat} \mid x = 0 \wedge v = 2]$ are not well-formed, as neither context corresponds to a valid manifest execution path. On the other hand, a well-formed type is allowed to include an error term in its denotation, e.g., type $[v: \text{nat} \mid \perp]$ is well-formed under type context $x:[v: \text{nat} \mid v > 0]$ as it always corresponds to a valid underapproximation.

Our second set of judgments defines a largely standard subtyping relation based on the underlying denotation of the types being related. In general, an overapproximate type and an underapproximate type are inhabited by incomparable sets of terms, even when they share a qualifier. A non-singleton underapproximate type can only be assigned to nondeterministic terms, while the corresponding overapproximate type can be given to many possible deterministic terms. On the other hand, the unconstrained generator for a base type, e.g., `int_gen()` or `tree_gen()`, can be checked against an underapproximate type qualified with any predicate, but it can only be assigned the overapproximate type qualified with $\top$. As a consequence, our subtyping relation does not relate overapproximate and underapproximate types, and our typing rules tightly control when one can be treated as another.

The disjunction rule (DISJUNCTION), which was informally introduced in [Sec. 2](#), merges the coverage types found along distinct control paths. Intuitively, the type $[v: \text{nat} \mid v = 1 \vee v = 2]$ is the disjunction of the types $[v: \text{nat} \mid v = 1]$ and $[v: \text{nat} \mid v = 2]$. Notice that only an inhabitant of both $[v: \text{nat} \mid v = 1]$ and $[v: \text{nat} \mid v = 2]$ should be included in their disjunction: e.g., the term $1 \oplus 2$ is one such inhabitant. Thus, we formally define this relation as the *intersection* of the denotations of two types.

The rules of our type system are defined in [Fig. 7](#), and include typing judgments for terms ($\Gamma \vdash e : \tau$), primitive operators ($\Gamma \vdash op : \tau$), and data constructors ($\Gamma \vdash d : \tau$). All our typing rules assume that terms are well-typed according to the basic type system; the basic type system is completely standard and we omit it here. The rules collectively maintain the invariant that terms can only be assigned a well-formed type. The rules for constants (T_{CONST}), operators (T_{OP}), and data constructors (T_{DT}) are straightforward. It relies on an auxiliary function, $\mathbf{Ty}$, to assign types to the primitives of λ^U . [Table 2](#) presents some examples of the typings provided by $\mathbf{Ty}$. We use uninterpreted predicates in the types of constructors: the types for list constructors, for example, use *emp*, *hd* and *tl*, to precisely capture that `[]` constructs an empty list, and that `(Cons x y)` builds a list containing *x* as its head and *y* as its tail.

The typing rules for function abstraction (T_{FUN}) and error (T_{ERR}) are similarly straightforward. In a λ -abstraction, the type of the bound variable (τ_x) is explicitly given. The error

⁵The definition of a type's denotation is given in [Sec. 5](#).

Typing

$$\boxed{\Gamma \vdash e : \tau \quad \Gamma \vdash op : \tau \quad \Gamma \vdash d : \tau}$$

$$\begin{array}{c}
\frac{\Gamma \vdash^{\text{WF}} [v: b \mid \perp]}{\Gamma \vdash \text{err} : [v: b \mid \perp]} \text{TErr} \quad \frac{\Gamma \vdash^{\text{WF}} \text{Ty}(c)}{\Gamma \vdash c : \text{Ty}(c)} \text{TConst} \quad \frac{\Gamma \vdash^{\text{WF}} \text{Ty}(d)}{\Gamma \vdash d : \text{Ty}(d)} \text{TDT} \quad \frac{\Gamma \vdash^{\text{WF}} \text{Ty}(op)}{\Gamma \vdash op : \text{Ty}(op)} \text{TOp} \\
\\
\frac{\Gamma \vdash^{\text{WF}} [v: b \mid v = x]}{\Gamma \vdash x : [v: b \mid v = x]} \text{TVARBASE} \quad \frac{\Gamma(x) = (a: \tau_a \rightarrow \tau_b) \quad \Gamma \vdash^{\text{WF}} a: \tau_a \rightarrow \tau_b}{\Gamma \vdash x : (a: \tau_a \rightarrow \tau_b)} \text{TVARFUN} \\
\\
\frac{\Gamma, x: \tau_x \vdash e : \tau \quad \Gamma \vdash^{\text{WF}} x: \tau_x \rightarrow \tau}{\Gamma \vdash \text{fun } (x: \tau_x) \rightarrow e : (x: \tau_x \rightarrow \tau)} \text{TFUN} \\
\\
\frac{\Gamma, x: \{v: b \mid \phi\}, f: (x: \{v: b \mid v \prec x \wedge \phi\} \rightarrow \tau) \vdash e : \tau \quad \Gamma \vdash x: \{v: b \mid \phi\} \rightarrow \tau}{\Gamma \vdash \text{fix } (f: (x: \{v: b \mid \phi\} \rightarrow \tau)) \text{ fun } (x: \{v: b \mid \phi\}) \rightarrow e : x: \{v: b \mid \phi\} \rightarrow \tau} \text{TFIX} \\
\\
\frac{\emptyset \vdash \tau <: \tau' \quad \emptyset \vdash e : \tau \quad \Gamma \vdash^{\text{WF}} \tau'}{\Gamma \vdash e : \tau'} \text{TSUB} \quad \frac{\Gamma \vdash \tau' <: \tau \quad \Gamma \vdash \tau <: \tau' \quad \Gamma \vdash e : \tau \quad \Gamma \vdash^{\text{WF}} \tau'}{\Gamma \vdash e : \tau'} \text{TEQ} \\
\\
\frac{\Gamma \vdash e : \tau_1 \quad \Gamma \vdash e : \tau_2 \quad \Gamma \vdash \tau_1 \vee \tau_2 = \tau \quad \Gamma \vdash^{\text{WF}} \tau}{\Gamma \vdash e : \tau} \text{TMERGE} \quad \frac{\Gamma \vdash e_x : \tau_x \quad \Gamma, x: \tau_x \vdash e : \tau \quad \Gamma \vdash^{\text{WF}} \tau}{\Gamma \vdash \text{let } x = e_x \text{ in } e : \tau} \text{TLETE} \\
\\
\frac{\Gamma \vdash op : \overline{a_i: \{v: b_i \mid \phi_i\}} \rightarrow \tau_x \quad \forall i, \Gamma \vdash v_i : [v: b_i \mid \phi_i] \quad \Gamma, x: \tau_x [\overline{a_i \mapsto v_i}] \vdash e : \tau \quad \Gamma \vdash^{\text{WF}} \tau}{\Gamma \vdash \text{let } x = op \overline{v_i} \text{ in } e : \tau} \text{TAPPOP} \quad \frac{\Gamma \vdash v_1 : (\tau_1 \rightarrow \tau_2) \rightarrow \tau_x \quad \Gamma \vdash v_2 : \tau_1 \rightarrow \tau_2 \quad \Gamma, x: \tau_x \vdash e : \tau \quad \Gamma \vdash^{\text{WF}} \tau}{\Gamma \vdash \text{let } x = v_1 v_2 \text{ in } e : \tau} \text{TAPPFUN} \\
\\
\frac{\Gamma \vdash v_1 : a: \{v: b \mid \phi\} \rightarrow \tau_x \quad \Gamma \vdash v_2 : [v: b \mid \phi] \quad \Gamma, x: \tau_x [a \mapsto v_2] \vdash e : \tau \quad \Gamma \vdash^{\text{WF}} \tau}{\Gamma \vdash \text{let } x = v_1 v_2 \text{ in } e : \tau} \text{TAPP} \quad \frac{\Gamma \vdash v : \tau_v \quad \Gamma \vdash^{\text{WF}} \tau \quad \Gamma, \overline{y: \tau_y} \vdash d_i(\overline{y}) : \tau_v \quad \Gamma, \overline{y: \tau_y} \vdash e_i : \tau}{\Gamma \vdash (\text{match } v \text{ with } \overline{d_i \overline{y} \rightarrow e_i}) : \tau} \text{TMATCH}
\end{array}$$

Fig. 7. Typing rules

Table 2. Example typings for λ^U primitives

Constants	$\text{Ty}(\text{true}) = [v: \text{bool} \mid v]$ $\text{Ty}(8) = [v: \text{nat} \mid v = 8] \dots$
Data	$\text{Ty}([]) = [v: b \text{ list} \mid \text{emp}(v)]$
Constructors	$\text{Ty}(\text{Cons}) = x: \{v: b \mid \top\} \rightarrow y: \{v: b \text{ list} \mid \top\} \rightarrow [v: b \text{ list} \mid \text{hd}(v, x) \wedge \text{tl}(v, y)] \dots$
Operators	$\text{Ty}(\text{nat_gen}) = [v: \text{unit} \mid \top] \rightarrow [v: \text{int} \mid 0 \leq v]$ $\text{Ty}(+) = x: \{v: \text{int} \mid \top\} \rightarrow y: \{v: \text{int} \mid \top\} \rightarrow [v: \text{int} \mid v = x + y] \dots$

term can be assigned an arbitrary bottom coverage base type. The variable rule (TVARBASE) establishes that the variable x in the type context with a base type can also be typed with the tautological qualifier $v = x$ (the rule's well-formedness assumption ensures that x is not free). This judgment allows us to, for example, type the function $\lambda x : \text{nat}. x$ with the type $x: \{v: \text{nat} \mid \top\} \rightarrow [v: \text{nat} \mid v = x]$, indicating that the return value is guaranteed to be exactly equal to the input x . Observe that the type of x under the type context $x: \{v: \text{nat} \mid \top\}$

(generated by the function rule TFUN) is *not* $[v: \text{nat} \mid \top]$. We cannot simply duplicate the qualifier for x from the type context here, as this is only sound when types characterize an overapproximation of program behavior. As an example, $\{v: \text{nat} \mid \top\}$ is a subtype of $\{v: \text{nat} \mid v = x\}$ under the type context $x: \{v: \text{nat} \mid \top\}$; in our underapproximate coverage type system, in contrast, $[v: \text{nat} \mid \top]$ is not a subtype of $[v: \text{nat} \mid v = x]$ under the type context $x: \{v: \text{nat} \mid \top\}$.

In a call-by-value language, a variable must refer to the same value throughout its scope; this property extends to the guarantees provided by its coverage type. Consider the following example:

```
⊢ let x = int_gen () in let y = 3 / x in x : [v: int | v = 0]
```

Although x may be bound to any integer, this program will never evaluate to 0, as the evaluation of the second `let` expression will raise a division-by-zero exception when x is 0. Our type system enforces the invariant that the coverage type of a variable is fixed when it is bound, ensuring that the variable provides consistent coverage guarantees whenever it is subsequently used. In the example above, to ensure safe execution, `int_gen ()` must be assigned a coverage type that excludes 0, i.e., $[v: \text{int} \mid v \neq 0]$. To achieve this, our subsumption rule (TSUB) is restricted to closed terms, permitting us to strengthen the coverage guarantee only when a closed generator like `int_gen ()` is introduced; and disallowing any coverage changes of variable x at the places where it is used.

The typing rule for application (TAPP) requires both its underapproximate argument type and the overapproximate parameter type to have the same qualifier, and furthermore requires that the type of the body (τ) is well-formed under the original type context Γ , enforcing that x (the result of the application) does not appear free in τ . When argument and parameter qualifiers are not identical, a subsumption rule is typically used to bring the two types into alignment. Recall the following example from [Sec. 2](#), suitably modified to conform to λ^U 's syntax:

```
bst_gen : lo: {v: int | ⊤} → hi: {v: int | lo ≤ v} → [v: int tree | ...], low: {v: int | ⊤} ⊢
  let (g: unit → int) = int_gen in let (x: unit) = () in
  let (high: int) = g x in let (y: int tree) = bst_gen low high in y
```

Here, the type of `high`, $[v: \text{int} \mid \top]$ is stronger than the type expected for the second parameter of `bst_gen`, $[v: \text{int} \mid \text{lo} \leq v]$. The subsumption rule (TSUB), that would normally allow us to strengthen the type of `high` to align with the required parameter type, is applicable to only closed terms, which `high` is not. For the same reason, we cannot use TSUB to strengthen the type of `high` when it is bound to `g x`. Thankfully, we can strengthen `g` when it is bound to `int_gen`: according to [Table 2](#), the operator `int_gen` has type $\{v: \text{unit} \mid \top\} \rightarrow [v: \text{int} \mid \top]$ and is also closed, and can thus be strengthened via TSUB , allowing us to type the call to `bst_gen` under the following, stronger type context:

```
bst_gen : lo: {v: int | ⊤} → hi: {v: int | lo ≤ v} → [v: int tree | ...], low: {v: int | ⊤},
g : [v: unit | ⊤][v: int | low ≤ v], x : [v: unit | ⊤], high : [v: int | low ≤ v] ⊢
  let (y: int tree) = bst_gen low high in y
```

The subsumption rule allows us to use `int_gen` in a context that requires *fewer* guarantees than `int_gen` actually provides, namely those values of `high` required by the signature of `bst_gen`. Intuitively, since our notion of coverage types records feasible executions in the type context in the form of existentials that serve as witnesses to an underapproximation, the strengthening provided by the subsumption rule establishes an invariant that all bindings

introduced into a type context only characterize valid behaviors in a program execution. When coupled with **TMERGE**, this allows us to *split* a typing derivation into multiple plausible strengthenings when a variable is introduced into the typing context and then *combine* the resulting types to reason about multiple feasible paths.

Now, using **TAPP** to type `bst_gen low high`, and **TVARBASE** to type the body of the `let` gives us:

```
bst_gen : lo : {v: int |  $\top$ }  $\rightarrow$  hi : {v: int |  $\text{lo} \leq v$ }  $\rightarrow$  [v: int tree | ...],
low : {v: int |  $\top$ }, high : [v: int |  $\text{low} \leq v$ ],
y : [v: int tree |  $\text{bst}(v) \wedge \forall u, \text{mem}(v, u) \implies \text{lo} < u < \text{hi}$ ][lo  $\mapsto$  low][hi  $\mapsto$  high]
 $\vdash$  y : [v: int tree |  $v = y$ ]
```

Observe that **TVARBASE** types the body as: `[v: int tree |  $v = y$ ]`, which is not closed. To construct a well-formed term, we need a formula equivalent to this type that accounts for the type of `y` in the current type context. The **TEQ** rule allows us to interchange formulae that are equivalent under a given type context to ensure the well-formedness of the types constructed. Unlike **TSUB**, it simply changes the form of a type's qualifiers, *without altering* the scope of feasible behaviors under the current context. In this example, such an equivalent closed type, given the binding for `y` in the type context under which the expression is being type-checked, would be:

```
let (y: int tree) = bst_gen low high in y :
[v: int tree |  $\exists y, (\text{bst}(y) \wedge \forall u, \text{mem}(y, u) \implies \text{low} < u < \text{high}) \wedge v = y$ ]
```

With these pieces in hand, we can see that the typing rule for `match` is a straightforward adaptation of the components we have already seen, where the type of matched variable v is assumed to have been strengthened by the rule **TSUB** to fit the type required to take the i th branch $\Gamma, \overline{y:\tau_y} \vdash d_i(\overline{y}) : \tau_v$. We can also safely assume the type of the branch τ_i is closed under original type context Γ , relying on **TEQ** to meet this requirement. While **TMATCH** only allows for a single branch to be typechecked, applying **TMERGE** allows us to reason about the coverage provided by multiple branches, which have all been typed according to this rule.

As in **TFUN**, the typing rule for recursive functions (**TFIX**) requires the types of the self-reference to f and the function parameter x to be explicitly given. Since types in our language serve as witnesses to feasible executions, the result type of any recursive procedure must characterize the set of values the procedure can plausibly return. Thus, the **TFIX** rule forces its first argument to always decrease according to some well-founded relation $\prec$. To see why we impose this restriction, consider the function **loop**:

```
let rec loop (n: nat) = loop n
```

Without our termination check, this function can be assigned the following type:

$$\{v: \text{nat} \mid \top\} \rightarrow [v: \text{nat} \mid v = 3]$$

despite the fact that this function never returns 3—or any value at all! The body of this expression can be type-checked under the following type context (via **TFIX** and **TFUN**):

$$n: \{v: \text{nat} \mid \top\}, \text{loop}: (n: \{v: \text{nat} \mid \top\} \rightarrow [v: \text{nat} \mid v = 3]) \vdash \text{loop } n : [v: \text{nat} \mid v = 3]$$

This judgment reflects an infinitely looping execution, however. Indeed, the same reasoning allows us to type this function with any result type. Constraining `loop`'s argument type to be decreasing according to $\prec$ yields the following typing obligation:

$$n: \{v: \text{nat} \mid \top\}, \text{loop}: (n: \{v: \text{nat} \mid v \prec n\} \rightarrow [v: \text{nat} \mid v = 3]) \vdash n : [v: \text{nat} \mid v = n]$$

where the qualifiers $\nu \prec \mathbf{n}$ and $\nu = \mathbf{n}$ conflict, raising a type error, and preventing `loop` from being recursively applied to $\mathbf{n}$.

5 Type soundness

Type denotations. Assuming a standard typing judgement for basic types, $\emptyset \vdash_{\mathbf{t}} e : t$, a type denotation for a type τ , $\llbracket \tau \rrbracket$, is a set of closed expressions:

$$\begin{aligned} \llbracket \{v:b \mid \phi\} \rrbracket &\doteq \{v \mid \emptyset \vdash_{\mathbf{t}} v : b \wedge \phi[v \mapsto v]\} \\ \llbracket [v: b \mid \phi] \rrbracket &\doteq \{e \mid \emptyset \vdash_{\mathbf{t}} e : b \wedge \forall v:b, \phi[v \mapsto v] \implies e \hookrightarrow^* v\} \\ \llbracket x:\tau_x \rightarrow \tau \rrbracket &\doteq \{f \mid \emptyset \vdash_{\mathbf{t}} f : [\tau_x \rightarrow \tau] \wedge \forall v_x \in \llbracket \tau_x \rrbracket \implies f v_x \in \llbracket \tau[x \mapsto v_x] \rrbracket\} \end{aligned}$$

In the case of an overapproximate refinement type, $\{v:b \mid \phi\}$, the denotation is simply the set of all values of type b whose elements satisfy the type's refinement predicate (ϕ), when substituted for all free occurrences of v in ϕ .⁶ Dually, the denotation of an underapproximate coverage type is the set of expressions that evaluate to v whenever $\phi[v \mapsto v]$ holds, where ϕ is the type's refinement predicate. Thus, every expression in such a denotation serves as a witness to a feasible, type-correct, execution. The denotation for a function type is defined in terms of the denotations of the function's argument and result in the usual way, ensuring that our type denotation is a logical predicate.

Type denotations under a type context. The denotation of a refinement type τ under a type context Γ (written $\llbracket \tau \rrbracket_{\Gamma}$) is:^{7,8}

$$\begin{aligned} \llbracket \tau \rrbracket_{\emptyset} &\doteq \llbracket \tau \rrbracket \\ \llbracket \tau \rrbracket_{x:\tau_x, \Gamma} &\doteq \{e \mid \forall v_x \in \llbracket \tau_x \rrbracket, \text{let } x = v_x \text{ in } e \in \llbracket \tau[x \mapsto v_x] \rrbracket_{\Gamma[x \mapsto v_x]}\} && \text{if } \tau \equiv \{v:b \mid \phi\} \\ \llbracket \tau \rrbracket_{x:\tau_x, \Gamma} &\doteq \{e \mid \exists \hat{e}_x \in \llbracket \tau_x \rrbracket, \forall e_x \in \llbracket \tau_x \rrbracket, \\ &\quad \text{let } x = e_x \text{ in } e \in \bigcap_{\hat{e}_x \hookrightarrow^* v_x} \llbracket \tau[x \mapsto v_x] \rrbracket_{\Gamma[x \mapsto v_x]}\} && \text{otherwise} \end{aligned}$$

The denotation of an overapproximate refinement type under a type context is mostly unsurprising, other than our presentation choice to use a `let`-binding, rather than substitution, to construct the expressions included in the denotations.

For a coverage type, however, the definition precisely captures our notion of a reachability witness by explicitly constructing an execution path as a sequence of `let`-bindings that justifies the inhabitant of the target type τ . Using `let`-bindings forces expressions in the denotation to make consistent choices when evaluated. The existential introduced in the definition captures the notion of an underapproximation, while the use of set intersection allows us to reason about non-determinism introduced by primitive generators like `nat_gen ()`.

Example 5.1. The term $x + 1$ is included in the denotation of the following type:

$$[v: \text{nat} \mid v = x + 1 \vee v = x + x]$$

⁶The denotation of an overapproximate refinement type is more generally $\{e:b \mid \emptyset \vdash e : b \wedge \forall v:b, e \hookrightarrow^* v \implies \phi[x \mapsto v]\}$. However, because such types are only used for function parameters, and our language syntax only admits values as arguments, our denotation uses the simpler form.

⁷In the last case, since $\hat{e}_x$ may nondeterministically reduce to multiple values, we employ intersection (not union), similar to the `DISJUNCTION` rule.

⁸When reasoning about a subset relation between the denotations of two types under a type context $\llbracket [v: b \mid \phi_1] \rrbracket_{\Gamma} \subseteq \llbracket [v: b \mid \phi_2] \rrbracket_{\Gamma}$ we require that the denotations be computed using the same Γ ; details are provided in the technical report [71].

under the type context $x : [\nu : \text{nat} \mid \nu = 1]$. This is justified by picking 1 for $\hat{e}_x$, which yields a set intersection that is equivalent to $\llbracket [\nu : \text{nat} \mid \nu = 2] \rrbracket$. Observe that *any* expression in $\llbracket [\nu : \text{nat} \mid \nu = 1] \rrbracket$, e.g. $0 \oplus 1$ and $1 \oplus 2$, yields an expression, **let** $x = 0 \oplus 1$ **in** $x + 1$ or **let** $x = 1 \oplus 2$ **in** $x + 1$, included in this intersection.

Example 5.2. On the other hand, the term x is not a member of the denotation of the type $[\nu : \text{nat} \mid \nu = x + 1 \vee \nu = x + x]$. To see why, let us pick `nat_gen()` for $\hat{e}_x$. This yields a set intersection that is equivalent to $\llbracket [\nu : \text{nat} \mid \top] \rrbracket$. While specific choices for e_x , e.g., `nat_gen()`, are included in this denotation, it does not work for all terms $e_x \in \llbracket [\nu : \text{nat} \mid \nu = 1] \rrbracket$. As one example, $0 \oplus 1 \oplus 2$ is an element of this set, but **let** $x = 0 \oplus 1 \oplus 2$ **in** x is clearly not a member of $\llbracket [\nu : \text{nat} \mid \top] \rrbracket$. Suppose instead that we picked a more restrictive expression for $\hat{e}_x$, like the literal 1 from the previous example. Here, it is easy to choose $e_x \in \llbracket [\nu : \text{nat} \mid \nu = 1] \rrbracket$ (e.g., the literal 1) such that **let** $x = e_x$ **in** $x \notin \llbracket [\nu : \text{nat} \mid \nu = 2] \rrbracket$.

Our main soundness result establishes the correctness of type-checking in the presence of coverage types with respect to a type's denotation:

Theorem 5.3. [Type Soundness] For all type contexts Γ , terms e and coverage types τ , $\Gamma \vdash e : \tau \implies e \in \llbracket \tau \rrbracket_\Gamma$.

It immediately follows that a closed input generator e with coverage type $[\nu : b \mid \phi]$ must produce every value satisfying ϕ , as desired.

6 Typing algorithm

The declarative typing rules are highly nondeterministic, relying on a combination of the `TMERGE` and `TSUB` rules to both explore and combine the executions needed to establish the desired coverage properties. In addition, each of the auxiliary typing relations depends on logical properties of the semantic interpretation of types. Any effective type checking algorithm based on these rules must address both of these issues. Our solution to the first problem is to implement a bidirectional type checker [11] whose type synthesis phase characterizes a set of feasible paths and whose type checking phase ensures those paths produce the desired results. Our solution to the second is to encode the logical properties into a *decidable* fragment of first order logic that can be effectively discharged by an SMT solver.

6.1 Bidirectional typing algorithm

As is standard in bidirectional type systems, our typing algorithm consists of a type synthesis judgement ($\Gamma \vdash e \Rightarrow \tau$) and a type checking judgement ($\Gamma \vdash e \Leftarrow \tau$) as presented in Fig. 8. Most of the rules are similar to the declarative typing rules, e.g., the type synthesis rule `SYNVARBASE` has the same premise as the declarative typing rule `TVARBASE` shown in Fig. 7. We use one type checking rule `CHKMATCH` and one synthesis rule `SYNAPPBASE` as examples to illustrate how we convert declarative typing rules into a typing algorithm.

Typing match. As we saw in Sec. 4, applying the declarative typing rule for `match` expressions typically requires first using several other rules to get things into the right form: `TMERGE` is required to analyze and combine the types of each branch, `TSUB` is used to equip each branch with the right typing context, and `TEQ` is used to remove any local or pattern variables from the type of a branch. Our bidirectional type system combines all of these into the single

Type Synthesis	$\boxed{\Gamma \vdash e \Rightarrow \tau}$	Type Check	$\boxed{\Gamma \vdash e \Leftarrow \tau}$
$\frac{\Gamma \vdash^{\text{WF}} \mathbf{Ty}(c)}{\Gamma \vdash c \Rightarrow \mathbf{Ty}(c)} \text{ SYNCONST}$	$\frac{\Gamma \vdash^{\text{WF}} [v: b \mid \perp]}{\Gamma \vdash \mathbf{err} \Rightarrow [v: b \mid \perp]} \text{ SYNERR}$	$\frac{\Gamma \vdash^{\text{WF}} [v: b \mid v = x]}{\Gamma \vdash x \Rightarrow [v: b \mid v = x]} \text{ SYNVARBASE}$	
$\frac{\Gamma(x) = a:\tau_a \rightarrow \tau_b \quad \Gamma \vdash^{\text{WF}} a:\tau_a \rightarrow \tau_b}{\Gamma \vdash x \Rightarrow a:\tau_a \rightarrow \tau_b} \text{ SYNVARFUN}$		$\frac{\Gamma, x:\{v: b \mid \top\} \vdash e \Rightarrow \tau \quad \Gamma \vdash^{\text{WF}} x:\{v: b \mid \top\} \rightarrow \tau}{\Gamma \vdash \lambda x:b.e \Rightarrow x:\{v: b \mid \top\} \rightarrow \tau} \text{ CHKFUN}$	
$\frac{\Gamma \vdash v_1 \Rightarrow \tau_2 \rightarrow \tau_x \quad [\tau_2] \notin b \quad \Gamma \vdash v_2 \Leftarrow \tau_2 \quad \Gamma' = x:\tau_x \quad \Gamma, \Gamma' \vdash e \Rightarrow \tau \quad \tau' = \mathbf{Ex}(\Gamma', \tau) \quad \Gamma \vdash^{\text{WF}} \tau'}{\Gamma \vdash \mathbf{let} x = v_1 v_2 \mathbf{in} e \Rightarrow \tau'} \text{ SYNAPPFUN}$		$\frac{\Gamma \vdash v_1 \Rightarrow a:\{v: b \mid \phi\} \rightarrow \tau_x \quad \Gamma' = a:[v: b \mid v = v_2 \wedge \phi], x:\tau_x \quad \Gamma, \Gamma' \vdash e \Rightarrow \tau \quad \tau' = \mathbf{Ex}(\Gamma', \tau) \quad \Gamma \vdash^{\text{WF}} \tau'}{\Gamma \vdash \mathbf{let} x = v_1 v_2 \mathbf{in} e \Rightarrow \tau'} \text{ SYNAPPBASE}$	
$\frac{\mathbf{Ty}(op) \Rightarrow \overline{a_i:\{v: b_i \mid \phi_i\}} \rightarrow \tau_x \quad \Gamma' = a_i:[v: b_i \mid v = v_i \wedge \phi_i], x:\tau_x \quad \Gamma, \Gamma' \vdash e \Rightarrow \tau \quad \tau' = \mathbf{Ex}(\Gamma', \tau) \quad \Gamma \vdash^{\text{WF}} \tau'}{\Gamma \vdash \mathbf{let} x = op \overline{v_i} \mathbf{in} e \Rightarrow \tau'} \text{ SYNAPPOP}$		$\frac{\Gamma \vdash e_x \Rightarrow \tau_x \quad \Gamma' = x:\tau_x \quad \Gamma, \Gamma' \vdash e \Rightarrow \tau \quad \tau' = \mathbf{Ex}(\Gamma', \tau) \quad \Gamma \vdash^{\text{WF}} \tau'}{\Gamma \vdash \mathbf{let} x = e_x \mathbf{in} e \Rightarrow \tau} \text{ SYNLETE}$	
$\frac{\forall i, \mathbf{Ty}(d_i) = \overline{y:\{v: b_y \mid \theta_y\}} \rightarrow [v: b \mid \psi_i] \quad \Gamma'_i = \overline{y:[v: b_y \mid \theta_y], a:[v: b \mid v = v_a \wedge \psi_i]} \quad \Gamma, \Gamma'_i \vdash e_i \Rightarrow \tau_i \quad \tau'_i = \mathbf{Ex}(\Gamma'_i, \tau_i) \quad \Gamma \vdash^{\text{WF}} \mathbf{Disj}(\overline{\tau'_i})}{\Gamma \vdash \mathbf{match} v_a \mathbf{with} \overline{d_i \bar{y} \rightarrow e_i} \Rightarrow \mathbf{Disj}(\overline{\tau'_i})} \text{ SYNMATCH}$			
$\frac{\emptyset \vdash e \Rightarrow \tau \quad \Gamma \vdash \tau <: \tau' \quad \Gamma \vdash^{\text{WF}} \tau'}{\Gamma \vdash e \Leftarrow \tau'} \text{ CHKSUB}$		$\frac{\Gamma, x:\tau_x \vdash e \Leftarrow \tau \quad \Gamma \vdash^{\text{WF}} x:\tau_x \rightarrow \tau}{\Gamma \vdash \lambda x:[\tau_x].e \Leftarrow (x:\tau_x \rightarrow \tau)} \text{ CHKFUN}$	
$\frac{\forall i, \mathbf{Ty}(d_i) = \overline{y:\{v: b_y \mid \theta_y\}} \rightarrow [v: b \mid \psi_i] \quad \Gamma'_i = \overline{y:[v: b_y \mid \theta_y], a:[v: b \mid v = v_a \wedge \psi_i]} \quad \Gamma, \Gamma'_i \vdash e_i \Rightarrow \tau_i \quad \tau'_i = \mathbf{Ex}(\Gamma'_i, \tau_i) \quad \Gamma \vdash \mathbf{Disj}(\overline{\tau'_i}) <: \tau' \quad \Gamma \vdash^{\text{WF}} \tau'}{\Gamma \vdash \mathbf{match} v_a \mathbf{with} \overline{d_i \bar{y} \rightarrow e_i} \Leftarrow \tau'} \text{ CHKMATCH}$			
	$\frac{\Gamma \vdash \lambda x:b.\lambda f:(b \rightarrow [\tau]).e \Leftarrow (x:\{v: b \mid \phi\} \rightarrow f:(x:\{v: b \mid v \prec x \wedge \phi\} \rightarrow \tau) \rightarrow \tau) \quad \Gamma \vdash^{\text{WF}} x:\{v: b \mid \phi\} \rightarrow \tau}{\Gamma \vdash \mathbf{fix} f:(b \rightarrow [\tau]).\lambda x:b.e \Leftarrow (x:\{v: b \mid \phi\} \rightarrow \tau)} \text{ CHKFIX}$		

Fig. 8. Bidirectional typing rules

CHKMATCH rule shown in Fig. 8. At a high level, this rule synthesizes a type for all the branches and then ensures that, in combination, they cover the desired type.

Similarly to other refinement type systems, when synthesizing the type for the branch for constructor d_i , we use a ghost variable $a:[v: b \mid v = v_a \wedge \psi_i]$ to ensure that the types of any pattern variables $\bar{y}$ are consistent with the parameters of d_i . This strategy allows us to avoid having to apply T_{SUB} to focus on a particular branch: instead, we simply infer a type for *each* branch, and then combine them using our disjunction operation. In order for the inferred type of a branch to make sense, we need to remove any occurrences of pattern variables or the ghost variable a . To do, we use the **Ex** function, which intuitively allows us to embed information from the typing context into a type. This function takes as input a typing context Γ and type τ and produces an equivalent type in which pattern and ghost variables do not

appear free. Finally, **CHKMATCH** uses **Disj** to ensure that the combination of the types of all the branches cover the required type $\Gamma \vdash \text{Disj}(\overline{\tau'_i}) <: \tau$.

Example 6.1. Consider how we might check that the body of the generator for even numbers introduced in [Sec. 2](#) has the expected type $[v: \text{int} \mid v \bmod 2 = 0]$:⁹

```
int_gen: {v: unit |  $\top$ }  $\rightarrow$  [v: int |  $\top$ ]  $\vdash$ 
  let (n: int) = int_gen() in let (b: bool) = n mod 2 == 0 in
  match b with true -> n | false -> err  $\Leftarrow$  [v: int |  $v \bmod 2 = 0$ ]
```

Our typing algorithm first adds the local variable **n** and **b** to the type context, and then checks the pattern-matching expression against the given type:

```
int_gen: {v: unit |  $\top$ }  $\rightarrow$  [v: int |  $\top$ ], n: [v: int |  $\top$ ], b: [v: bool |  $v \iff n \bmod 2 = 0$ ]  $\vdash$ 
  match b with true -> n | false -> err  $\Leftarrow$  [v: int |  $v \bmod 2 = 0$ ]
```

The **CHKMATCH** rule first synthesizes types for the two branches separately. Inferring a type of the second branch using the existing type context:

```
..., b: [v: bool |  $v \iff n \bmod 2 = 0$ ], b': [v: bool |  $v = b \wedge \neg v$ ]  $\vdash$  err  $\Rightarrow$  [v: int |  $\perp$ ]
```

adds a ghost variable **b'** to reflect the fact that $n \bmod 2$ must not be equal to 0 in this branch. By next applying the **TErr** rule, our algorithm infers the type $[v: \text{int} \mid \perp]$ for this branch. The rule next uses **Ex** to manifest **b'** in the inferred type, encoding the path constraints under which this type holds (i.e. *b* is false).

```
..., b: [v: bool |  $v \iff n \bmod 2 = 0$ ], b': [v: bool |  $v = b \wedge \neg v$ ]  $\vdash$ 
  err  $\Rightarrow$  [v: int |  $\exists b', b' = b \wedge \neg b' \wedge \perp$ ]
```

Thus, the synthesized type for the second branch is $[v: \text{int} \mid \neg b \wedge \perp]$ modulo some trivial simplification. The type of the first branch provides a better demonstration of why **Ex** is needed:

```
..., b: [v: bool |  $v \iff n \bmod 2 = 0$ ], b': [v: bool |  $v = b \wedge v$ ]  $\vdash$ 
  n  $\Rightarrow$  [v: int |  $v = n$ ]
```

After applying **Ex**, the inferred type is $[v: \text{int} \mid \exists b', b' = b \wedge b' \wedge v = n]$; after simplification, this becomes $[v: \text{int} \mid b \wedge v = n]$. The disjunction of these two types:

$$\text{Disj}([v: \text{int} \mid b \wedge v = n], [v: \text{int} \mid \neg b \wedge \perp]) = [v: \text{int} \mid (b \wedge v = n) \vee (\neg b \wedge \perp)]$$

results in exactly the type shown in [Sec. 2](#). This can then be successfully checked against the target type $[v: \text{int} \mid v \bmod 2 = 0]$.

Application. Our type synthesis rules for function application adopt a strategy similar to **CHKMATCH**'s, trying to infer the strongest type possible for an expression that uses the result of a function application. The rule for a function whose parameter is an overapproximate refinement type (**SYNAPPBASE**) is most interesting, since it has to bridge the gap with an argument that has an underapproximate coverage type. When typing *e*, the expression that uses the result of the function call, the rule augments the typing context with a ghost variable *a*. This variable records that the coverage type of the argument must overlap with the type expected by the function (both must satisfy the refinement predicate ϕ): if this intersection is empty, i.e., the type of *a* is equivalent to $\perp$, we will fail to infer a type for *e*, as no type will be well-formed in this context. As with **CHKMATCH**, **SYNAPPBASE** uses **Ex** to ensure that it does not infer a type that depends on *a*.

⁹We have replaced the **if** from the original example with a **match** expression, to be consistent with the syntax of λ^U .

6.2 Auxiliary typing functions

Algorithm 1: Subtyping query encoding

Inputs : A type context Γ , and two base coverage types $[v: b \mid \phi_1]$ and $[v: b \mid \phi_2]$.

Output : A FOL formula that can be automatically checked by an SMT solver.

```

1 Procedure Subtyping( $\Gamma, [v: b \mid \phi_1], [v: b \mid \phi_2]$ ) :=
2   match  $\Gamma$ :
3     case  $\emptyset$  do
4       return  $\forall v: b, \phi_2 \implies \phi_1$ ;
5     case  $\Gamma, x: (a: \tau_a \rightarrow \tau)$  do
6        $\phi \leftarrow \text{Subtyping}(\Gamma, [v: b \mid \phi_1], [v: b \mid \phi_2]);$ 
7       return  $\phi$ ;
8     case  $\Gamma, x: \{v: b_x \mid \phi_x\}$  do
9        $\tau_1 \leftarrow [v: b \mid \forall x: b_x, \phi_x[v \mapsto x] \implies \phi_1];$ 
10       $\tau_2 \leftarrow [v: b \mid \forall x: b_x, \phi_x[v \mapsto x] \implies \phi_2];$ 
11      return Subtyping( $\Gamma, [v: b \mid \tau_1], [v: b \mid \tau_2]$ );
12     case  $\Gamma, x: [v: b_x \mid \phi_x]$  do
13        $\tau_1 \leftarrow [v: b \mid \exists x: b_x, \phi_x[v \mapsto x] \wedge \phi_1];$ 
14        $\tau_2 \leftarrow [v: b \mid \exists x: b_x, \phi_x[v \mapsto x] \wedge \phi_2];$ 
15       return Subtyping( $\Gamma, [v: b \mid \tau_1], [v: b \mid \tau_2]$ );

```

The auxiliary **Disj** and **Ex** operations are straightforward syntactic transformations; their full definitions can be found in the technical report [71]. More interesting is how we algorithmically check well-formedness and subtyping. Our type checking algorithm translates both obligations into logical formulae that can be discharged by a SMT solver. Both obligations are encoded by the **Subtyping** subroutine shown in Algo 1. **Subtyping**($\Gamma, [v: b \mid \phi_1], [v: b \mid \phi_2]$) encodes the bindings in Γ in the typing context from right to left, before checking whether ϕ_1 implies ϕ_2 . Variables with function types, on the other hand, are omitted entirely, as qualifiers cannot have function variables in FOL. Variables with an overapproximate (underapproximate) type are translated as a universally (existential) quantified variable, and are encoded into the refinement of both coverage types.

Example 6.2. Consider the subtyping obligation generated by Example 6.1 above:

$$\text{int_gen}:\{v: \text{unit} \mid \top\} \rightarrow [v: \text{int} \mid \top], n:[v: \text{int} \mid \top], b:[v: \text{bool} \mid v \iff n \bmod 2 = 0] \vdash \\ [v: \text{int} \mid (b \wedge v = n) \vee (\neg b \wedge \perp)] <: [v: \text{int} \mid v \bmod 2 = 0]$$

$$\begin{array}{c}
\frac{\not\models \text{Subtyping}(\Gamma, [v: b \mid \perp], [v: b \mid \phi])}{\text{err} \notin \llbracket [v: b \mid \phi] \rrbracket_\Gamma} \quad \frac{\models \text{Subtyping}(\Gamma, [v: b \mid \phi_1], [v: b \mid \phi_2])}{\Gamma \vdash [v: b \mid \phi_1] <: [v: b \mid \phi_2]} \\
\\
\frac{\models \text{Subtyping}(\Gamma, [v: b \mid \phi_2], [v: b \mid \phi_1])}{\Gamma \vdash \{v: b \mid \phi_1\} <: \{v: b \mid \phi_2\}}
\end{array}$$

Fig. 9. Auxiliary algorithmic typing functions

This obligation is encoded by the following call to **Subtyping**

$$\begin{aligned}
& \text{Subtyping}(\text{int_gen}:\{v: \text{unit} \mid \top\} \rightarrow [v: \text{int} \mid \top], n:[v: \text{int} \mid \top], b:[v: \text{bool} \mid v \iff n \bmod 2 = 0], \\
& \quad [v: \text{int} \mid (b \wedge v = n) \vee (\neg b \wedge \perp)], [v: \text{int} \mid v \bmod 2 = 0]) \\
& \equiv \text{Subtyping}(\text{int_gen}:\{v: \text{unit} \mid \top\} \rightarrow [v: \text{int} \mid \top], n:[v: \text{int} \mid \top], \\
& \quad [v: \text{int} \mid \exists b, b \iff n \bmod 2 = 0 \wedge (b \wedge v = n) \vee (\neg b \wedge \perp)], \\
& \quad [v: \text{int} \mid \exists b, b \iff n \bmod 2 = 0 \wedge v \bmod 2 = 0]) \\
& \equiv \text{Subtyping}(\text{int_gen}:\{v: \text{unit} \mid \top\} \rightarrow [v: \text{int} \mid \top], \\
& \quad [v: \text{int} \mid \exists n, \top \wedge \exists b, b \iff n \bmod 2 = 0 \wedge (b \wedge v = n) \vee (\neg b \wedge \perp)], \\
& \quad [v: \text{int} \mid \exists n, \top \wedge \exists b, b \iff n \bmod 2 = 0 \wedge v \bmod 2 = 0]) \\
& \equiv \text{Subtyping}(\emptyset, [v: \text{int} \mid \exists n, \top \wedge \exists b, b \iff n \bmod 2 = 0 \wedge (b \wedge v = n) \vee (\neg b \wedge \perp)], \\
& \quad [v: \text{int} \mid \exists n, \top \wedge \exists b, b \iff n \bmod 2 = 0 \wedge v \bmod 2 = 0]) \\
& \equiv \forall v, \exists n, \top \wedge \exists b, b \iff n \bmod 2 = 0 \wedge v \bmod 2 = 0 \implies \\
& \quad \exists n, \top \wedge \exists b, b \iff n \bmod 2 = 0 \wedge (b \wedge v = n) \vee (\neg b \wedge \perp)
\end{aligned}$$

This is equivalent to formula (2) from Sec. 2:

$$\forall v, (v \bmod 2 = 0) \implies (\exists n, \exists b, b \iff n \bmod 2 = 0 \wedge (b \wedge v = n) \vee (\neg b \wedge \perp))$$

Using **Subtyping**, it is straightforward to discharge well-formedness and subtyping obligations using the rules shown in Fig. 9. In the case of **WFBASE**, for example, observe that the error term **err** is always an inhabitant of the type $[v: b \mid \perp]$ for arbitrary base type b . Thus, to check the last assumption of **WFBASE**, it suffices to iteratively check if any coverage types in the type context are a supertype of their associated bottom type.

Discharging subtyping obligations is slightly more involved, as we need to ensure that the formulas sent to the SMT solver are decidable. Observe that in order to produce effectively decidable formulas, the encoding strategy realized by **Subtyping** always generates a formula of the form $\forall \bar{x}. \exists \bar{y}. \phi$, i.e. it does not allow for arbitrary quantifier alternations. To ensure that this is sound strategy, we restrict all overapproximate refinement types in a type context to not have any free variables that have a coverage type. This constraint allows us to safely lift all universal quantifiers to the top level, thus avoiding arbitrary quantifier alternations.

As an example of a scenario disallowed by this restriction, consider the following type checking judgment:

$$x:[v: \text{nat} \mid v > 0] \vdash \lambda y : \text{nat}. x + y \Leftarrow y:\{v: \text{nat} \mid v > x + 1\} \rightarrow [v: \text{nat} \mid \phi]$$

This judgment produces the following subtyping check:

$$x:[v: \text{nat} \mid v > 0], y:\{v: \text{nat} \mid v > x + 1\} \vdash [v: \text{nat} \mid v = x + y] <: [v: \text{nat} \mid \phi]$$

```

1  val union: (unit -> 'a) -> (unit -> 'a) -> (unit -> 'a)
2  let union g1 g2 = fun () -> (g1 () ⊕ g2 ())
3
4  val pos_gen: unit -> int (* positive number generator *)
5  val neg_gen: unit -> int (* negative number generator *)
6
7  let bool_gen = union (fun () -> true) (fun () -> false)
8  let nat_gen = union pos_gen (fun () -> 0)
9  let int_gen = union neg_gen nat_gen

```

Fig. 10. A union combinator example.

where the normal refinement type $\{v: \text{nat} \mid v > x + 1\}$ in the type context has free variable x that has coverage type. Evaluating this judgment entails solving the formula:

$$\forall v, (\exists x, x > 0 \wedge (\forall y, y > x + 1 \implies \phi)) \implies (\exists x, x > 0 \wedge (\forall y, y > x + 1 \implies v = x + y))$$

which is not decidable due to the quantifier alternation $\forall v \exists x \forall y$.

Theorem 6.3. [Soundness of Algorithmic Typing] For all type context Γ , term e and coverage type τ , $\Gamma \vdash e \Leftarrow \tau \implies \Gamma \vdash e : \tau$

Theorem 6.4. [Completeness of Algorithmic Typing] Assume an oracle for all formulas produced by the **Subtyping** subroutine. Then for any type context Γ , term e and coverage type τ , $\Gamma \vdash e : \tau \implies \Gamma \vdash e \Leftarrow \tau$.

7 Polymorphic coverage types

Most mainstream PBT frameworks provide a set of combinators that enable users to compositionally build test generators. For example, the QCheck framework provides a **union** combinator shown in Fig. 10 that produces a new generator which covers the combined test cases of two test input generators. Unlike $\oplus$ which takes two base values as arguments, the **union** combinator combines two input generators, defined as function values. As shown in Fig. 10, the **union** combinator is polymorphic and cannot be assigned a (principal) type in the monomorphic coverage type system we have discussed so far. As a consequence, we need to extend this type system in order to, e.g., type the simple generator on lines 7-9.

To support this flexibility, we extend this type system to align with how OCaml implements polymorphism. Thus, a polymorphic term is assigned a *principal* type (i.e., a most general type) that includes universally quantified type variables like 'a.

Example 7.1. In this extended type system, the non-deterministic choice operator $\oplus$ is now assigned the following polymorphic function coverage type:

$$\tau_{\oplus} \doteq x:\{v: 'a \mid \top\} \rightarrow y:\{v: 'a \mid \top\} \rightarrow [v: 'a \mid v = x \vee v = y]$$

The type $\tau_{\oplus}$ imposes no constraint on its two parameters x and y , and guarantees the return value (v) must cover both (i.e., $v = x \vee v = y$).

While type polymorphism is sufficient to capture the basic behavior of generator combinators, it falls short in a type system, where we aim to specify and provide coverage guarantees. Consider lines 8-9 in Fig. 10: the **union** combinator combines a generator that produces only positive integers (**pos_gen**) with the constant generator that always returns 0 (**fun () → 0**),

to build a natural number generator (`nat_gen`). It is then used again to create an integer generator (`int_gen`) from a negative number generator (`neg_gen`) and the aforementioned natural number generator. Thus, `union` takes test generators with *arbitrary* coverage guarantees and builds a generator whose coverage guarantee combines those guarantees.

Our goal is to assign the `union` combinator a principal coverage type τ_{union} that reflects that it builds a generator that returns a superset of the values produced by its arguments. This guarantee is similar to that of the non-deterministic choice operator $\oplus$ shown in Example 7.1; but because the `union` combinator takes functions as parameters, the analog of the qualifier “ $\nu = x \vee \nu = y$ ” for the `union` combinator cannot be expressed in our current qualifier language. One potential solution is to add union and intersection types (e.g., $\tau \sqcap \tau$ and $\tau \sqcup \tau$) into our type system, but this strategy will not work for more complicated combinators like `bind`, `frequency`, and `fix` shown in Fig. 2.

To address this, we draw inspiration from other variants of refinement type systems (Kaki and Jagannathan [30] and Vazou et al. [61]) and introduce *qualifier polymorphism* to allow types to be parameterized over *predicate variables*. With this extension, the principal type of τ_{union} becomes:

$$\tau_{\text{union}} \doteq \forall P_1 P_2 : 'a \rightarrow \text{bool}. (\text{unit} \rightarrow [\nu: 'a \mid P_1(\nu)]) \rightarrow (\text{unit} \rightarrow [\nu: 'a \mid P_2(\nu)]) \rightarrow (\text{unit} \rightarrow [\nu: 'a \mid P_1(\nu) \vee P_2(\nu)])$$

Here, P_1 and P_2 are universally quantified predicate variables of type $'a \rightarrow \text{bool}$. This type states that “for all possible qualifiers $P_1(\nu)$ and $P_2(\nu)$, if the two input generators cover $P_1(\nu)$ and $P_2(\nu)$ respectively, then the generator that results from combining them with a union operation covers $P_1(\nu) \vee P_2(\nu)$ ”.

Much like type variables, predicate variables must appear at the front of the type and be instantiated with concrete qualifiers during function application. For instance, when applying the `union` combinator on line 8 in Fig. 10, we instantiate the predicate variables according to the following assignment:

$$P_1 \mapsto \lambda \nu. 0 < \nu \quad \text{and} \quad P_2 \mapsto \lambda \nu. \nu = 0$$

This allows us to assign the `union` combinator the following coverage type:

$$\tau_{\text{union}_1} \doteq (\text{unit} \rightarrow [\nu: \text{int} \mid 0 < \nu]) \rightarrow (\text{unit} \rightarrow [\nu: \text{int} \mid \nu = 0]) \rightarrow (\text{unit} \rightarrow [\nu: \text{int} \mid 0 \leq \nu])$$

The rest of the program can be checked using this type, which aligns the types of `neg_gen` and `fun () → 0`. Our extended bidirectional typing rules can automatically infer predicate variable assignments from the qualifier in the corresponding arguments using a simple form of syntactic unification, which we discuss below.

7.1 Extended type syntax

The syntax of our extended type system is presented in Fig. 11, with new additions highlighted in green.

Basic types. Following the conventions of standard polymorphic type systems [50], the basic type system of λ^U supports both type variables ($'a$) and type quantification ($'a.t$). The type variable $'a$ in our type system can be used in refinement types $[\nu: 'a \mid \phi]$, and thus can only be instantiated with base type (b). As in OCaml, type and predicate quantifiers are

Type Variable	$'a, 'b, 'c, \dots$
Base Types	$b ::= 'a \mid \text{unit} \mid \text{bool} \mid \text{nat} \mid \text{int} \mid b \text{ list} \mid b \text{ tree} \mid \dots$
Basic Types	$t ::= b \mid t \rightarrow t \mid 'a.t$
Predicate Variable	$P, \dots$
Predicates	$\text{pred} ::= P \mid \overline{\lambda x}. \phi$
Qualifier	$\phi ::= l \mid \perp \mid \top \mid \text{op}(\overline{l}) \mid \text{up}(\overline{x}) \mid \neg \phi \mid \phi \wedge \phi \mid \phi \vee \phi \mid \phi \implies \phi$ $\mid \forall u:b. \phi \mid \exists u:b. \phi \mid \text{pred}(\overline{l})$
Refinement Types	$\tau ::= [v: b \mid \phi] \mid \{v: b \mid \phi\} \mid x:\tau \rightarrow \tau \mid 'a.\tau \mid \forall P: (\overline{b} \rightarrow \text{bool}). \tau$
Type context	$\Gamma ::= \emptyset \mid \Gamma, x:\tau \mid \Gamma, P:t \mid \Gamma, 'a$

Fig. 11. Extended λ^U syntax.

omitted when the context is clear, e.g., $'a.\forall P:'a \rightarrow \text{bool}.\text{unit} \rightarrow [v: 'a \mid P(v)]$ is written as $\text{unit} \rightarrow [v: 'a \mid P(v)]$.

Qualifiers and refinement types. In addition to supporting type quantification (e.g., $'a.\tau$), coverage refinement types also allow quantified *predicate variables* P . As is standard, *predicates* are functions (e.g., $\overline{\lambda x}.\phi$) whose parameters must be base type arguments and which return boolean values ($\overline{b} \rightarrow \text{bool}$). Fully applied predicate variables may appear in qualifiers, like $P(\overline{l})$, analogous to primitive operators (op). Predicate application $(\overline{\lambda x}.\phi)(\overline{l})$ is simplified as $\phi[\overline{x} \mapsto \overline{l}]$. Notably, the extension of the qualifier language yields both universal (for type-checking polymorphic functions) and existential (for type-checking polymorphic function application) quantified type variables and predicate variables. Our subtyping rules instantiate these variables such that derived verification conditions only contain universal quantified variables. Type and predicate variables are encoded as *uninterpreted sorts* and *uninterpreted predicates* in the verification conditions generated by our type system, ensuring that type-checking remains within the decidable fragment of first order logic.

Type context and type erasure function. To support these extensions, the type context Γ is extended to include type variables ($\Gamma, 'a$) as well as predicate variables ($\Gamma, P:t$). The updated type erasure function $\llbracket \tau \rrbracket$ preserves all quantified type variables and discards all quantified predicate variables.

7.2 Extended typing rules

The introduction of polymorphism induces the three new sets of typing rules shown in Fig. 12. The additional well-formedness $\text{WF}_{\text{POLYTYPE}}$ and $\text{WF}_{\text{POLYPRED}}$ rules ensure that refinement types are in prenex normal form, i.e., all quantifiers appear at the front of types. The subtyping extensions are also straightforward: the $\text{SUB}_{\text{POLYTYPE}}$ rule permits instantiating type variables with any valid base type, while the $\text{SUB}_{\text{POLYPRED}}$ rule allows a predicate variable P to be instantiated with any well-formed predicate $\overline{\lambda x}.\phi$. Finally, we have two new declarative typing rules (TPOLYTYPE and TPOLYPRED) that move type and predicates quantifiers from the type into the type context.

Well-Formedness

$$\begin{array}{c}
\boxed{\Gamma \vdash^{\text{WF}} t \quad \Gamma \vdash^{\text{WF}} \tau \quad \vdash^{\text{WF}} \Gamma} \\
\\
\frac{\text{FreeTypeVar}(t) \subseteq \Gamma}{\Gamma \vdash^{\text{WF}} t} \text{WFBASICTYPE} \quad \frac{\vdash^{\text{WF}} \Gamma \quad [\Gamma], v:b \vdash_t \phi : \text{bool}}{\Gamma \vdash^{\text{WF}} b} \text{WFBASE} \\
\\
\frac{\Gamma, \bar{a} \vdash^{\text{WF}} \tau \quad \text{no type quantification in } \tau}{\Gamma \vdash^{\text{WF}} \bar{a}.\tau} \text{WFPOLYTYPE} \quad \frac{\Gamma \vdash^{\text{WF}} [\nu: b \mid \phi] \quad \Gamma, P:\bar{b} \rightarrow \text{bool} \vdash^{\text{WF}} \tau \quad \text{no predicate quantification in } \tau}{\Gamma \vdash^{\text{WF}} \forall P:\bar{b} \rightarrow \text{bool}.\tau} \text{WFPOLYPRED}
\end{array}$$

Subtyping

$$\frac{\Gamma \vdash^{\text{WF}} \tau[a \mapsto b]}{\Gamma \vdash 'a.\tau <: \tau[a \mapsto b]} \text{SUBPOLYTYPE} \quad \frac{\Gamma, x:\bar{b} \vdash^{\text{WF}} \phi}{\Gamma \vdash \forall P:\bar{b} \rightarrow \text{bool}.\tau <: \tau[P \mapsto \bar{\lambda}x.\phi]} \text{SUBPOLYPRED} \quad \boxed{\Gamma \vdash \tau_1 <: \tau_2}$$

Typing

$$\frac{\Gamma, 'a \vdash e : \tau \quad \Gamma \vdash^{\text{WF}} \tau}{\Gamma \vdash e : 'a.\tau} \text{TPOLYTYPE} \quad \frac{\Gamma, P:\bar{b} \rightarrow \text{bool} \vdash e : \tau \quad \Gamma \vdash^{\text{WF}} \tau}{\Gamma \vdash e : \forall P:\bar{b} \rightarrow \text{bool}.\tau} \text{TPOLYPRED} \quad \boxed{\Gamma \vdash e : \tau}$$

Fig. 12. Extended typing relations

Type denotation and soundness. The extended denotational semantics for types is:

$$\begin{aligned}
\llbracket 'a.\tau \rrbracket &\doteq \bigcup_b \llbracket \tau[a \mapsto b] \rrbracket \quad \text{where } b \text{ contains no type variables} \\
\llbracket \forall P:(\bar{b} \rightarrow \text{bool}).\tau \rrbracket &\doteq \bigcup_{\phi} \llbracket \tau[P \mapsto \bar{\lambda}x.\phi] \rrbracket \quad \text{where } x:\bar{b} \vdash_t \phi : \text{bool}
\end{aligned}$$

The fundamental theorem and overall type soundness remain unchanged.

7.3 Extended typing algorithm**Type Synthesis**

$$\boxed{\Gamma \vdash e \Rightarrow \tau}$$

Type Check

$$\boxed{\Gamma \vdash e \Leftarrow \tau}$$

$$\frac{\Gamma, 'a \Leftarrow e : \tau \quad \Gamma \vdash^{\text{WF}} \tau}{\Gamma \vdash e \Leftarrow 'a.\tau} \text{CHKPOLYTYPE} \quad \frac{\Gamma, p:\bar{b} \rightarrow \text{bool} \Leftarrow e : \tau \quad \Gamma \vdash^{\text{WF}} \tau}{\Gamma \vdash e \Leftarrow \forall p:\bar{b} \rightarrow \text{bool}.\tau} \text{CHKPOLYPRED}$$

$$\frac{\Gamma \vdash v_1 \Rightarrow \tau_1 \quad \Gamma \vdash v_2 \Rightarrow \tau_2 \quad [\tau_2] \notin b \quad \text{Inst}(\Gamma, \tau_1, \tau_2) = a:\tau'_2 \rightarrow \tau_x \quad \Gamma \vdash \tau_2 <: \tau'_2 \quad \Gamma' = x:\tau_x \quad \Gamma, \Gamma' \vdash e \Rightarrow \tau \quad \tau' = \text{Ex}(\Gamma', \tau) \quad \Gamma \vdash^{\text{WF}} \tau'}{\Gamma \vdash \text{let } x = v_1 \ v_2 \text{ in } e \Rightarrow \tau'} \text{SYNAPPFUN}$$

$$\frac{\Gamma \vdash v_1 \Rightarrow \tau_1 \quad \Gamma \vdash v_2 \Rightarrow [\nu: b \mid \phi_2] \quad \text{Inst}(\Gamma, \tau_1, [\nu: b \mid \phi_2]) = a:\{\nu: b \mid \phi\} \rightarrow \tau_x \quad \Gamma' = a:[\nu: b \mid \nu = v_2 \wedge \phi], x:\tau_x \quad \Gamma, \Gamma' \vdash e \Rightarrow \tau \quad \tau' = \text{Ex}(\Gamma', \tau) \quad \Gamma \vdash^{\text{WF}} \tau'}{\Gamma \vdash \text{let } x = v_1 \ v_2 \text{ in } e \Rightarrow \tau'} \text{SYNAPPBASE}$$

Fig. 13. Extended bidirectional typing rules

The extended bidirectional typing algorithm is shown in Fig. 13. We introduce two new rules **CHKPOLYTYPE** and **CHKPOLYPRED** to type-check polymorphic terms, which are similar to **TPOLYTYPE** and **TPOLYPRED** from Fig. 12. More challenging is function application,

Algorithm 2: Predicate variable instantiation

Inputs : A type context Γ , a function type $\forall \overline{P}.t.x:\tau_{\text{param}} \rightarrow \tau_{\text{ret}}$, and an argument type τ_2
Output : the instantiated function type

- 1 **Procedure** **Inst**($\Gamma, \forall \overline{P}.t.x:\tau_{\text{param}} \rightarrow \tau_{\text{ret}}, \tau_2$) :=
- 2 $\overline{P}:t \leftarrow \overline{P}:t \setminus \text{FreePredVar}(\tau_{\text{param}});$
- 3 $\sigma \leftarrow \text{UnifyType}(\Gamma, \emptyset, [(\tau_{\text{param}}, \tau_2)]);$
- 4 $x:\tau_{\text{param}} \rightarrow \tau_{\text{ret}} \leftarrow \sigma(x:\tau_{\text{param}} \rightarrow \tau_{\text{ret}});$
- 5 **return** $x:\tau_{\text{param}} \rightarrow (\forall \overline{P}.t.\tau_{\text{ret}});$

which in the declarative type system relies on the auxiliary `SUBPOLYTYPE` and `SUBPOLYPRED` subtyping rules to instantiate both type variables and predicate variables in polymorphic refinement types. We modify the previous typing algorithm for function application (i.e., `SYNAPPFUN` and `SYNAPPBASE`) as shown in Fig. 13, where modified parts are colored in gray. These two rules first infer a function type (v_1) and type for the corresponding argument (v_2), then invoke the instantiation subroutine **Inst**, which instantiates the polymorphic function type with respect to the argument type (e.g., instantiates τ_1 on τ_2 in the rule `SYNAPPFUN`). Since we perform type synthesis instead of type checking for the function v_1 in `SYNAPPFUN`, we additionally inline the rule `CHKSUB` to ensure the argument type τ_2 is a subtype of the instantiated parameter type τ'_2 . The output of the **Inst** subroutine also ensures that the resulting parameter type (e.g., $\{v: b \mid \phi\}$ in `SYNAPPBASE`) contains no free predicate variables. This guarantees that the remaining typing rules can be directly copied from the original typing algorithm.

As shown in Algorithm 2, the **Inst** subroutine assumes that type qualifiers (e.g., 'a. τ) have been eliminated, and divides the input function type τ_1 into three parts: quantified predicates $\overline{P}:t$, parameter type τ_{param} , and the return type τ_{ret} . Newly quantified predicates are updated as shown on line 2 to guarantee that all predicate variables appearing in τ_{param} (i.e., $\text{FreePredVar}(\tau_{\text{param}})$) are instantiated. The algorithm then uses the unification subroutine to obtain a predicate assignment σ that ensures predicate variables within the parameter type τ_{param} are consistent with the argument type τ_2 (line 3). After substituting predicate variables with the corresponding assignments (line 4), **Inst** subroutine then returns an instantiated function type (line 5) whose return type is properly quantified.

The unification algorithm is shown in Algorithm 3 and consists of two subroutines `UnifyType` and `UnifyQualifier` that collectively unify refinement types and qualifiers in a manner similar to the standard type unification algorithm [50]. The `UnifyType` subroutine takes an initial predicate assignment solution (σ) and a list of type constraints (l). When encountering a base refinement type containing a free predicate variable that is not included in the type context (line 5), the algorithm invokes the qualifier unification subroutine (line 6) to get a predicate assignment σ_q . It then unions the predicate assignment and continues to process the remaining constraints after substitution (lines 6 and 7). A function type is unified piecewise (lines 12–13); otherwise, a mismatched case leads to a unification error (line 15). The `UnifyQualifier` subroutine works similarly but recursively processes over the syntactic structure of the qualifier instead.

Example 7.2. As shown on line 5 in Fig. 10, type-checking the application `union neg_gen nat_gen` requires instantiating the quantified type predicates of τ_{union} based

Algorithm 3: Predicate unification

Inputs : A type context Γ , an initial predicate variable assignment σ , and a list of type alignment constraints l

Output : A predicate variable assignment that satisfies all input constraints.

```

1 Procedure UnifyType( $\Gamma, \sigma, l$ ) :=
2   match  $l$ :
3     case [] do return  $\sigma$  ;
4     case ( $\{v: b \mid \phi_1\}, \{v: b \mid \phi_2\} :: l$  or ( $[v: b \mid \phi_1], [v: b \mid \phi_2] :: l$ ) do
5       if  $\text{FreePredVar}(\phi_1) \setminus \text{Dom}(\Gamma) \neq \emptyset$  then
6          $\sigma_q = \text{UnifyQualifier}(\Gamma, \emptyset, [\phi_1, \phi_2])$ ;
7          $\sigma = \sigma_q(\sigma)$ ;
8          $l = \sigma_q(l)$ ;
9         return  $\text{UnifyType}(\Gamma, \sigma \cup \sigma_q, l)$ ;
10      else
11        return  $\text{UnifyType}(\Gamma, \sigma, l)$ ;
12    case ( $x: \tau_{11} \rightarrow \tau_{12}, x: \tau_{21} \rightarrow \tau_{22} :: l$ ) do
13      return  $\text{UnifyType}(\sigma, (\tau_{11}, \tau_{21}) :: (\tau_{21}, \tau_{22}) :: l)$ ;
14    case ( $\tau_1, \tau_2 :: l$ ) do
15      Raise Failure;

```

Inputs : A type context Γ , an initial predicate variable assignment σ , and a list of qualifier alignment constraints l

Output : A predicate variable assignment that satisfies all input constraints.

```

16 Procedure UnifyQualifier( $\Gamma, \sigma, l$ ) :=
17   match  $l$ :
18     case [] do return  $\sigma$  ;
19     case ( $P(\bar{x}), \phi :: l$ ) do
20        $\sigma_q = \{P \mapsto \bar{\lambda}x. \phi\}$ ;
21        $\sigma = \sigma_q(\sigma)$ ;
22        $l = \sigma_q(l)$ ;
23       return  $\text{UnifyQualifier}(\Gamma, \sigma \cup \sigma_q, l)$ ;
24     case ( $\phi_1 \vee \phi_2, \phi'_1 \vee \phi'_2 :: l$ ) do
25       return  $\text{UnifyQualifier}(\Gamma, \sigma, (\phi_1, \phi'_1) :: (\phi_2, \phi'_2) :: l)$ ;
26     case ( $\phi_1 \wedge \phi_2, \phi'_1 \wedge \phi'_2 :: l$ ) do
27       return  $\text{UnifyQualifier}(\Gamma, \sigma, (\phi_1, \phi'_1) :: (\phi_2, \phi'_2) :: l)$ ;
28     case ( $\phi_1 \implies \phi_2, \phi'_1 \implies \phi'_2 :: l$ ) do
29       return  $\text{UnifyQualifier}(\Gamma, \sigma, (\phi_1, \phi'_1) :: (\phi_2, \phi'_2) :: l)$ ;
30     case ( $\neg \phi, \neg \phi' :: l$ ) do
31       return  $\text{UnifyQualifier}(\Gamma, \sigma, (\phi, \phi') :: l)$ ;
32     case ( $\forall x. \phi, \forall x. \phi' :: l$ ) do
33       return  $\text{UnifyQualifier}(\Gamma, \sigma, (\phi, \phi') :: l)$ ;
34     case ( $\exists x. \phi, \exists x. \phi' :: l$ ) do
35       return  $\text{UnifyQualifier}(\Gamma, \sigma, (\phi, \phi') :: l)$ ;
36     case ( $\phi, \phi' :: l$ ) do
37       Raise Failure;

```

on its negative and natural number generator arguments.

$$\begin{aligned} \text{union} &: (\text{unit} \rightarrow [\nu: \text{int} \mid P_1(\nu)]) \rightarrow \\ & \quad (\text{unit} \rightarrow [\nu: \text{int} \mid P_2(\nu)]) \rightarrow (\text{unit} \rightarrow [\nu: \text{int} \mid P_1(\nu) \vee P_2(\nu)]), \\ \text{neg_gen} &: \text{unit} \rightarrow [\nu: \text{int} \mid \nu < 0], \text{nat_gen} : \text{unit} \rightarrow [\nu: \text{int} \mid 0 \leq \nu] \vdash \\ & \quad (\text{union neg_gen}) \text{ nat_gen} \Rightarrow ? \end{aligned}$$

We type check the inner application (union neg_gen) first. The rule `SYNAPPFUN` uses the `Inst` subroutine to unify the type $\text{unit} \rightarrow [\nu: \text{int} \mid P_1(\nu)]$ against $\text{unit} \rightarrow [\nu: \text{int} \mid \nu < 0]$. The predicate assignment after unification is $P_1 \mapsto \lambda \nu. \nu < 0$, and the synthesized type of the application (union neg_gen) is

$$(\text{unit} \rightarrow [\nu: \text{int} \mid P_2(\nu)]) \rightarrow (\text{unit} \rightarrow [\nu: \text{int} \mid \nu < 0 \vee P_2(\nu)])$$

Now we can type check the second application $(\text{union neg_gen}) \text{ nat_gen}$ in a similar way, where the `Inst` subroutine unifies the parameter type $\text{unit} \rightarrow [\nu: \text{int} \mid P_2(\nu)]$ against $\text{unit} \rightarrow [\nu: \text{int} \mid 0 \leq \nu]$. The predicate assignment after unification is $P_2 \mapsto \lambda \nu. 0 \leq \nu$, and the coverage type of the final application is $\text{unit} \rightarrow [\nu: \text{int} \mid \nu < 0 \vee 0 \leq \nu]$, which covers all integers.

Note that our instantiation algorithm is not complete, as it relies on the assumption that the syntactic structures of two types under unification are aligned. For example, the `Inst` subroutine cannot unify $[\nu: \text{int} \mid P_1(\nu) \vee P_2(\nu)]$ with $[\nu: \text{int} \mid \nu \geq 0]$, although one possible solution is assigning P_1 to $\lambda \nu. \nu > 0$ and P_2 to $\lambda \nu. \nu = 0$.

Theorem 7.3. [Soundness of Extended Algorithmic Typing] For all type contexts Γ , term e and coverage type τ , $\Gamma \vdash e \Leftarrow \tau \implies \Gamma \vdash e : \tau$

Theorem 7.4. [Completeness of Extended Algorithmic Typing] Assuming an oracle for all formulas produced by the `Subtyping` subroutine, and assuming `Inst` subroutine always succeeds, then for any type context Γ , term e and coverage type τ , $\Gamma \vdash e : \tau \implies \Gamma \vdash e \Leftarrow \tau$.

8 The coverage monad and monadic generator combinators

As discussed in [Sec. 1](#), real-world PBT frameworks like QCheck define the type of input generators as a monad and provide combinators for compositionally defining input generators. In order to align with this practice, this section defines a *coverage monad* that provides coverage guarantees about input generators defined in this style. This monad and the monadic generator combinators described in this section can all be desugared into the core language from the previous section.

8.1 Coverage monad

In real-world programming languages, computational nondeterminism is typically introduced via a primitive pseudorandom number generator (PRNG) that produces unpredictable sequences of numbers based on an random seed. This facility is used by PBT frameworks like the generator library `Gen` of QCheck to parameterize the behavior of generators by a PRNG state, allowing test executions to be replayed by reusing the same initial seed. [Fig. 14](#) shows the definition of `Gen` in QCheck. The type of a generator 'a `Gen.t` is a function type $\text{RS.t} \rightarrow \text{'a}$ (line 3), which takes a random seed and returns a generated test value of type 'a. The `return` operator returns a constant value, regardless of the input state (line 4); while the

```

1  module Gen = struct
2    (* RS.t is the internal state of PRNG *)
3    type 'a t = RS.t -> 'a
4    let return (x: 'a) = fun (st: RS.t) -> x
5    let bind (g: 'a t) (f: 'a -> 'b t) =
6      fun (st: RS.t) -> let (x: 'a) = g st in f x st
7  end

```

Fig. 14. The definition of the test generator monad in QCheck.

bind operator passes the PRNG state through both the generator and the bind function (lines 5–6), respectively.

Example 8.1. In QCheck, a generator for positive integers can be compositionally defined by using the monadic bind combinator to get a random value from the natural number generator `nat_gen` and replacing it with 1 when it is a non-positive integer:

```

1  val nat_gen: int Gen.t
2  let pos_gen: int Gen.t =
3    bind nat_gen (fun (n: int) ->
4      if 0 < n then return n else return 1)

```

$$\mathcal{M} [v: b \mid \phi] \doteq \{v: \text{RS.t} \mid \top\} \rightarrow [v: b \mid \phi]$$

$$\begin{aligned} \text{bind}^c \mathcal{M} [v: b_1 \mid \phi_1] (x: \{v: b_1 \mid \phi_2\} \rightarrow \mathcal{M} [v: b_2 \mid \phi_3]) &\doteq \\ \mathcal{M} [v: b_2 \mid \exists x: b_1. (\phi_1 \wedge \phi_2) [v \mapsto x] \wedge \phi_3] \end{aligned}$$

$$\text{return} : x: \{v: 'a \mid \top\} \rightarrow \mathcal{M} [v: 'a \mid v = x]$$

$$\text{bind} : \mathcal{M} [v: 'a \mid P_1(v)] \rightarrow (x: \{v: 'a \mid P_2(v)\} \rightarrow \mathcal{M} [v: 'b \mid P_3(x, v)]) \rightarrow$$

$$\text{bind}^c \mathcal{M} [v: 'a \mid P_1(v)] (x: \{v: 'a \mid P_2(v)\} \rightarrow \mathcal{M} [v: 'b \mid P_3(x, v)])$$

$$\equiv \mathcal{M} [v: 'b \mid \exists x. P_1(x) \wedge P_2(x) \wedge P_3(x, v)] \quad (\text{after unfolding } \text{bind}^c)$$

Fig. 15. The coverage monad and the signatures of its operators.

While `nat_gen` and `pos_gen` produce different sets of values, they both have the same basic type: `int Gen.t`. To better capture the coverage guarantees of these sorts of monadic generators, we define the coverage monad $\mathcal{M} [v: b \mid \phi]$, shown in Fig. 15. As expected, the shape of this monad augments the type parameter of `Gen.t` with a qualifier that captures its coverage properties. The definition of $\mathcal{M} [v: b \mid \phi]$ has two consequences: first, since only non-function values can be qualified, the coverage monad can only range over base types. Second, the coverage guarantees of a monadic generator cannot reference the random seed. This latter restriction makes intuitive sense, as a monadic test generator of type $\mathcal{M} [v: b \mid \phi]$ should be guaranteed to produce values satisfying predicate ϕ regardless of the random seed. We can use the coverage monad to capture the different coverages of `nat_gen` and `pos_gen`, as $\mathcal{M} [v: \text{int} \mid 0 \leq v]$ and $\mathcal{M} [v: \text{int} \mid 0 < v]$, respectively.

Fig. 15 also gives the signatures of the operators of the coverage monad; their definitions are the same as in Fig. 14. The signature of `return` reflects that it lifts an arbitrary value into

```

1  val power:  $m:\{v:\text{int} \mid 0 \leq v\} \rightarrow x:\{v:\text{int} \mid 0 \leq v\} \rightarrow [v:\text{int} \mid v = m^x]$ 
2  let power_gen = bind (int_bound 10:  $[v:\text{int} \mid v \leq 10]$ )
3    (fun (x:  $\{v:\text{int} \mid 0 \leq v\}$ ) -> return (power 2 x))

```

Fig. 16. A generator for powers of 2 less than 2^{10} .

a generator that always produces that value (i.e., $v = x$). The signature of the `bind` operator uses a special notation, bind^C , for its codomain; this alias will be helpful in the upcoming discussion of generator combinators, which are built using `bind`. Intuitively, the qualifier in this type introduces an existentially quantified variable x that represents the intermediate value produced by the first argument of `bind`. This value is required to satisfy both the coverage guarantee of the test generator (ϕ_1) as well as the safety constraint associated with the `bind` function (ϕ_2).

To understand the definition of bind^C , consider how the definition of `bind` in Fig. 14 can be typed using the polymorphic type system of the previous section. After unfolding bind^C , the initial typing goal is:

$$g:\mathcal{M}[v:'a \mid P_1(v)], f:(x:\{v:'a \mid P_2(v)\} \rightarrow \mathcal{M}[v:'b \mid P_3(x, v)]), \\ \vdash \text{fun } (st:\text{RS.t}) \rightarrow \text{let } x = g \text{ st in } f \ x \text{ st} : [v:'b \mid \exists x. P_1(x) \wedge P_2(x) \wedge P_3(x, v)]$$

After moving the PRNG state and the value produced by the first generator into the context, the typing goal is:

$$g:\mathcal{M}[v:'a \mid P_1(v)], f:(x:\{v:'a \mid P_2(v)\} \rightarrow \mathcal{M}[v:'b \mid P_3(x, v)]), s:\text{RS.t}, x:[v:'a \mid P_1(v)] \\ \vdash f \ x \ s : [v:'b \mid \exists x. P_1(x) \wedge P_2(x) \wedge P_3(x, v)]$$

Similarly to how `bst_gen` was typed in Sec. 4, we can use TEQ to remove the existential quantifier in the target type, and then use TSUB to bring x into alignment with the argument type of f to yield the expected type.

Example 8.2. The coverage guarantees of generators built from monadic combinators are also derived compositionally. As an example, consider the definition of a generator for powers of 2 from 2^0 to 2^{10} shown in Fig. 16. After randomly generating an integer using `int_bound 10`, `power_gen` raises 2 to this power and returns the result. Note that the coverage guarantees of `power` are only provided for arguments greater than 0, however, a fact that will be reflected in the final type of `power_gen`. Importantly, the instantiations of three predicate variables in the type of bind^C come directly from the qualifiers of its arguments:

$$P_1 \mapsto \underbrace{\lambda v. v \leq 10}_{\text{int_bound } 10} \quad P_2 \mapsto \underbrace{\lambda v. 0 \leq v}_{\text{fun } x:\{v:\text{int} \mid 0 \leq v\} \rightarrow} \quad P_3 \mapsto \underbrace{\lambda x. \lambda v. v = 2^x}_{\text{power } 2 \ x}$$

Allowing us to derive the desired coverage type for `power_gen`:

$$\text{power} : \dots \vdash \text{power_gen} : \mathcal{M}[v:\text{int} \mid \exists x. 0 \leq x \wedge x \leq 10 \wedge v = 2^x]$$

Over- and under-approximate monads. Monadic combinators also lend themselves well to an over-approximate analysis via traditional refinement types. In the context of test input generators, such specifications are useful in contexts where developers want to prioritize certain *safe* values that always meet the preconditions of the system under test, e.g., when probing boundary cases. As an example of this situation, consider the following generator:


```

1  let biased_nat_gen (st: RS.t) =
2    union (int_range 0 10) (return Int.max_int)
3
4  ⊢ biased_nat_gen:  $\mathcal{M} \{v: \text{int} \mid 0 \leq v \leq \text{Int.max\_int}\}$ 

```

Fig. 17. Example of safety monad.

```

1  let biased_nat_gen (st: RS.t) =
2    union (int_range 0 10) (return Int.max_int)
3
4  ⊢ biased_nat_gen:  $\mathcal{M} \{v: \text{int} \mid 0 \leq v \leq \text{Int.max\_int}\}$ 

```

`biased_nat_gen` does not enumerate all natural numbers; instead, it samples from a biased distribution that emphasizes small numbers (e.g., $[0, 10]$) and the maximum integer value (`Int.max_int`), the latter of which covers the critical edge case for overflow testing. To cover this scenario, we can similarly define a dual *safety* monad for generators $\mathcal{M} \{v: b \mid \phi\}$. Here, `biased_nat_gen` can be assigned the safety monad type $\mathcal{M} \{v: \text{int} \mid 0 \leq v \leq \text{Int.max_int}\}$, ensuring that all sampled values fall within this range. We further discuss real-world instances of such usage in [Sec. 9](#).

Using monad combinators to define generators enables the seamless use of both safety and coverage monads to provide richer specifications that capture their ability to produce every value that meets a precondition and only such values. Consider a generator g_1 for lists of integers which has the following type:

$$\vdash g_1 : \text{bind}^c \mathcal{M} \{v: \text{int} \mid 0 \leq v\} (n: \text{int} \rightarrow \mathcal{M} [v: \text{int list} \mid \text{len}(v, n) \wedge \text{sorted}(v)])$$

This type specifies that g_1 produces all possible sorted lists of integers with length n , where n is drawn from a generator assigned a safety monad type (e.g., `biased_nat_gen`) which is biased toward values believed to be important by developers. Conversely, consider a generator g_2 with the following type:

$$\vdash g_2 : \text{bind}^c \mathcal{M} [v: \text{int} \mid 0 \leq v] (n: \text{int} \rightarrow \mathcal{M} \{v: \text{int list} \mid \text{len}(v, n) \wedge \text{sorted}(v)\})$$

Here, g_2 produces all possible non-negative list lengths, while imposing no specific coverage obligations over the list contents of a generated sorted list of a given length. This form is useful when the target property under test is sensitive to shape of the datatype (i.e., list length) but not necessarily to the elements it contains. Developers are then allowed to specialize the generated list of length n as a fixed choice (e.g., $[1; 2; \dots; n]$), rather than generating all possible sorted lists, which would require more effort. In both of these examples, the combined use of safety and coverage monads provides developers more flexibility to specify generators that are neither totally over-approximated nor totally under-approximated.

8.2 Monadic combinator examples

This section provides examples of the coverage guarantees our system can provide for several monadic combinators typically found in PBT frameworks like QCheck. We show the (monadic) coverage type of each example combinator first using the coverage monad, and then a version in which $\mathcal{M}$ and `bindc` are unfolded.

```

option :  $\mathcal{M} [v: 'a \mid P(v)] \rightarrow$ 
   $\text{bind}^C \mathcal{M} [v: 'a \mid P(v)] (x: 'a \rightarrow \mathcal{M} [v: 'a \text{ option} \mid v = \text{None} \vee v = \text{Some}(x)])$ 
   $\equiv \mathcal{M} [v: 'a \text{ option} \mid v = \text{None} \vee \exists x. P(x) \wedge v = \text{Some}(x)]$  (after unfolding)

pair :  $\mathcal{M} [v: 'a \mid P_1(v)] \rightarrow \mathcal{M} [v: 'b \mid P_2(v)] \rightarrow$ 
   $\text{bind}^C \mathcal{M} [v: 'a \mid P_1(v)]$ 
   $(x: 'a \rightarrow \text{bind}^C \mathcal{M} [v: 'b \mid P_2(v)] (y: 'b \rightarrow \mathcal{M} [v: 'a \times 'b \mid v = (x, y)]))$ 
   $\equiv \mathcal{M} [v: 'a \times 'b \mid P_1(\text{fst}(v)) \wedge P_2(\text{snd}(v))]$  (after unfolding)

```

The `option` combinator lifts a generator into a version for option types that covers both the `Some()` and `None` cases ($v = \text{None} \vee v = \text{Some}(x)$). Similarly, the `pair` combinator returns a pair containing the results of two input test generators ($v = (x, y)$).

```

list_repeat :  $s: \{v: \text{int} \mid 0 \leq v\} \rightarrow g: (\mathcal{M} [v: 'a \mid P(v)] \rightarrow$ 
   $\mathcal{M} [v: 'a \text{ list} \mid \text{len}(v, s) \wedge \forall x. \text{mem}(v, x) \implies P(x)])$ 

```

The `list_repeat` combinator uses an input generator (g) to construct a list of s elements. Its coverage type guarantees that it produces all lists of the stipulated length $\text{len}(v, s)$ that contain elements covered by the input generators (i.e., $\forall x. \text{mem}(v, x) \implies P_2(x)$). As a simple example, the following generator produces all lists with three positive numbers:

```
1 let pos_list_gen = list_repeat 3 pos_gen
```

```

oneof :  $s: \{v: \text{int} \mid 0 < v\} \rightarrow (i: \{v: \text{int} \mid P_1(v)\} \rightarrow \mathcal{M} [v: 'a \mid P_2(i, v)]) \rightarrow$ 
   $\text{bind}^C (\mathcal{M} [v: \text{int} \mid 0 \leq v < s]) (i: \{v: \text{int} \mid P_1(v)\} \rightarrow \mathcal{M} [v: 'a \mid P_2(i, v)])$ 
   $\equiv \mathcal{M} [v: 'a \mid \exists i. 0 \leq i < s \wedge P_1(i) \wedge P_2(i, v)]$  (after unfolding)

frequency :  $s: \{v: \text{int} \mid 0 < v\} \rightarrow (i: \{v: \text{int} \mid P_1(v)\} \rightarrow \mathcal{M} [v: \text{int} \times 'a \mid P_2(i, v)]) \rightarrow$ 
   $\text{bind}^C (\text{bind}^C (\mathcal{M} [v: \text{nat} \mid 0 \leq v < s])$ 
   $(i: \{v: \text{int} \mid P_1(v)\} \rightarrow \mathcal{M} [v: \text{int} \times 'a \mid P_2(i, v)]))$ 
   $(p: \text{nat} \times 'a \rightarrow \mathcal{M} [v: 'a \mid \text{fst}(p) > 0 \wedge \text{snd}(p) = v])$ 
   $\equiv \mathcal{M} [v: 'a \mid \exists i. 0 \leq i < s \wedge P_1(i) \wedge \exists w. w > 0 \wedge P_2(i, (w, v))]$  (after unfolding)

```

The `oneof` combinator is a more powerful version of the `union` combinator that combines an arbitrary number of test generators, instead of only two. This combinator takes two arguments: the total number of input generators (s) and an indexed function (f) where $f\ i$ represents the i^{th} input generator.¹⁰ As captured by its type signature, the number of input generators is constrained by predicate variable P_1 and the coverage guarantee of each input generator is constrained by P_2 . The coverage type of `union` guarantees that it

¹⁰The `oneof` and `frequency` combinators in QCheck use lists instead of a function to hold its input generators. Our formulation of both combinators allows them to straightforwardly combine generators with different coverage guarantees, without having to deal with heterogeneous lists.

builds a generator that covers everything that the input generators do (i.e., $\exists i. 0 \leq i < s$). As an example, the following (redundant) generator for integers uses `oneof` to combine 3 generators: a negative number generator (`neg_gen`), a constant generator that only produces 0 (`return 0`), and a positive number generator (`pos_gen`).

```

1  let int_gen = oneof 3 (function
2      | 0 -> neg_gen
3      | 1 -> return 0
4      | _ -> pos_gen)

```

The `frequency` combinator is similar to `oneof`, but it additionally allows the developer to provide a weight for each input generator that governs their sampling distribution. The type of this combinator requires that each of these weights are positive ($w > 0$).

$$\text{fix} : (m:\text{nat} \rightarrow (n:\{v:\text{nat} \mid v < m\} \rightarrow \mathcal{M} [v:\text{'a} \mid P(n, v)])) \rightarrow \mathcal{M} [v:\text{'a} \mid P(m, v)] \rightarrow (m:\text{nat} \rightarrow \mathcal{M} [v:\text{'a} \mid P(m, v)])$$

The `fix` combinator constructs a test generator that produces larger test inputs from a function that generates smaller test inputs; the first parameter of this function is a natural number that limits the number of recursive calls. Unsurprisingly, this combinator has the same premises as `TFix` in Fig. 7.

Example 8.3. We demonstrate how these combinators work by reimplementing the tree generator example from Sec. 1 using the `fix` and `frequency` combinators. The new implementation is given in Fig. 18. Since every `tree` contains at least one element, the base case returns an error. Otherwise, if the target number of elements is 1 (line 5), we generate a leaf node by binding the natural number generator with the `Leaf` constructor. In the other cases, we use the `frequency` combinator to combine two test generators with weight 1 and 2 respectively (line 9 - 12). The first generator is a singleton tree generator, while the second generator calls `self` recursively to generate left and right subtrees containing half the nodes. Note that each subtree has at most half the target number of nodes, to ensure that the final tree contains the expected number of elements.

Although this tree generator accepts a bound n as input, it cannot be assigned the coverage type $n:\text{nat} \rightarrow \mathcal{M} [v:\text{nat tree} \mid \text{numElem}(v) \leq n]$, which would require it to generate all

```

1  type 'a tree = Leaf of 'a | Node of 'a tree * 'a tree
2  let tree_gen : int -> int tree Gen.t = fix
3      (fun n self -> match n with
4          | 0 -> return err
5          | 1 -> bind nat_gen (fun x -> return (Leaf x))
6          | n ->
7              frequency 2
8                  (function
9                      | 0 -> (1, bind nat_gen (fun x -> return (Leaf x)))
10                     | _ -> (2, bind (self n/2) (fun tr1 ->
11                         bind (self n/2) (fun tr2 ->
12                             return (Node (tr1, tr2))))))
13      )

```

Fig. 18. Defining a tree generator using `frequency` and `fix`.

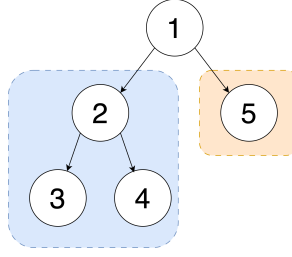


Fig. 19. A 5-node tree that cannot be produced by `tree_gen 5`.

trees containing at most n elements. This is because the implementation divides the input bound evenly during recursive calls (lines 10 and 11), making it unable to construct trees that are highly unbalanced. For instance, with input 5, the generator cannot produce the tree illustrated in Fig. 19, where the left subtree (highlighted in blue) contains 3 elements, exceeding the divided bound $5/2$. Instead, we can assign it the coverage type $n:\text{nat} \rightarrow \mathcal{M} [v:\text{nat tree} \mid 2^{\text{depth}(v)} \leq n]$, which captures the idea that all trees of depth m can be covered when the input bound is at least 2^m . According to this type, any tree can be generated by this tree generator given a sufficiently large input bound. This alternative interpretation of the input bound as “depth” is coverage-complete but less intuitive than the “number of nodes” interpretation, highlighting the intricate nature involved in underapproximate-style reasoning.

9 Implementation and evaluation

Implementation. We have implemented a coverage type checker, called *Poirot*, based on the above approach. *Poirot* targets functional, non-concurrent OCaml programs that rely on libraries to manipulate algebraic data types; it consists of approximately 11K lines of OCaml and uses Z3 [8] as its backend solver.

Poirot takes as input an OCaml program representing a test input generator and a user-supplied coverage type for that generator. After basic type-checking and translation into MNF, *Poirot* applies bidirectional type inference and checking to validate that the program satisfies the requirements specified by the type. Our implementation provides built-in coverage types for a number of OCaml primitives, including constants, various arithmetic operators, and data constructors for a range of datatypes. Refinements defined in coverage types can also use predefined (polymorphic) uninterpreted predicates that capture non-trivial datatype shape properties. For example, the uninterpreted predicate $\text{mem}(l, u)$ indicates the element u ’s a is contained in the data type instance l ’s a list; the uninterpreted predicate $\text{len}(l, 3)$ indicates the list l has length 3, or the tree l has depth 3. The semantics of these uninterpreted predicates are defined as a set of FOL-encoded lemmas and axioms to facilitate automated verification; e.g., the lemma $\text{len}(l, 0) \implies \forall u, \neg \text{mem}(l, u)$ indicates that the empty datatype instance contains no element.

In order to improve efficiency, verification conditions generated by *Poirot* are simplified before being solved by the underlying SMT solver. For example, the `Ex` function generates queries of the form $\exists x. x = v \wedge \phi$, which can be simplified to $\phi[x \mapsto v]$. This transformation can significantly reduce both the number of quantifiers and the overall size of queries.

Table 3. Experimental results on hand-written generators. Existing benchmarks are annotated with their source: QuickChick [36] (*), QuickCheck [5] (°), and bespoke test input generators from [34] (*) and [68] (°).

	#Branch	Rec	#LVar	#UP	#Query	(max. #V, #E)	total(avg. time) (ms)
SizedList*	3	✓	9	3	8	(2, 5)	10.45(1.31)
SortedList*	3	✓	13	4	11	(4, 7)	31.21(2.84)
UniqueList°	3	✓	10	5	9	(2, 5)	12.28(1.36)
SizedTree*	3	✓	12	4	11	(2, 7)	15.00(1.36)
CompleteTree*	2	✓	9	5	8	(2, 5)	12.01(1.50)
RedBlackTree*	6	✓	44	7	41	(5, 20)	83.28(2.03)
SizedBST*	4	✓	28	6	25	(6, 12)	42.69(1.71)
BatchedQueue°	1		5	1	5	(2, 4)	6.58(1.65)
BankersQueue°	1		5	1	5	(2, 4)	7.53(1.51)
Stream°	3	✓	10	4	9	(2, 6)	11.84(1.32)
SizedHeap°	4	✓	17	5	15	(5, 9)	23.10(1.54)
LeftistHeap°	2	✓	13	5	12	(2, 7)	47.06(3.92)
SizedSet°	3	✓	26	6	23	(5, 11)	45.17(1.96)
UnbalanceSet°	4	✓	35	6	31	(6, 14)	67.22(2.17)

9.1 Completeness of hand-written generators

We have evaluated Poirot on a corpus of hand-written, non-trivial test input generators drawn from a variety of sources (see Table 3). These benchmarks provide test input generators over a diverse range of datatypes, including various kinds of lists, trees, queues, streams, heaps, and sets. For each datatype implementation, Poirot type checks the provided implementation against its supplied coverage type to verify that the generator is able to generate all possible datatype instances consistent with this type. Our uninterpreted predicates allow us to capture non-trivial structural properties. For example, to verify a red-black tree generator, we use the predicate *black_height*(v, n) to indicate that all branches of the tree v have exactly n black nodes, the predicate *no_red_red*(v) to indicate v contains no red node with red children, and the predicate *root_color*(v, b) to indicate the root of the tree v has the red (black) color when the boolean value b is true (false).¹¹

Given this rich set of predicates, it is straightforward to express interesting coverage types. For example, given size s and lower bound lo , we can express the property that a sorted list generator *sorted_list_gen* must generate all possible sorted lists of length s and in which all elements are greater than or equal to lo , as the following type:

$$s:\{v:\text{int} \mid 0 \leq v\} \rightarrow lo:\{v:\text{int} \mid \top\} \rightarrow \\ [v:\text{int list} \mid \text{len}(v, s) \wedge \text{sorted}(v) \wedge \forall u, \text{mem}(v, u) \implies lo \leq u]$$

Notice that this type is remarkably similar to a normal refinement type:

$$s:\{v:\text{int} \mid 0 \leq v\} \rightarrow lo:\{v:\text{int} \mid \top\} \rightarrow \\ \{v:\text{int list} \mid \text{len}(v, s) \wedge \text{sorted}(v) \wedge \forall u, \text{mem}(v, u) \implies lo \leq u\}$$

albeit with the return type marked as a coverage type to capture our desired must-property.

The first group of columns in Table 3 describes the salient features of our benchmarks. Each benchmark exhibits non-trivial control-flow, containing anywhere from 1 to 6 nested branches; the majority of our benchmarks are also recursive (column Rec). The number of local (i.e., let-bound) variables (column #LVars) is a proxy for path lengths that must be

¹¹These uninterpreted predicates can be found in the implementation of the red-black tree generator given in [36].

```

type ty = Ty_nat | Ty_arr of ty * ty

type term =
  | Const of int
  | Var of int
  | Abs of ty * term
  | App of term * term

type tyctx = ty list

```

Fig. 20. Datatypes from the STLC case study.

encoded within the types inferred by our type-checker; column #UP indicates the number of uninterpreted predicates found in the benchmark’s type specification.

The second group of columns presents type checking results. The column #Query indicates the number of SMT queries that are triggered during type checking. The column #($\forall, \exists$) indicates the maximum number of universal and existential quantifiers in these queries, respectively. The $\exists$ column is a direct reflection of control-flow (path) complexity — complex generators with deeply nested match-expressions like `RedBlackTree` result in queries with over 20 existential quantifiers. These numbers broadly track with the values in columns #Branch and #LVar. Despite the complexity of some of these queries, as evidenced by the number of their quantifiers, overall verification time (average verification time per query, resp.), reported in the last column, is quite reasonable, with times ranging from 6.58 to 83.28 milliseconds, i.e., all benchmarks finishing in less than 0.1 second.

9.2 Case study: well-typed STLC terms

We have also applied Poirot to a more substantial example: a generator for well-typed simply typed lambda calculus (STLC) terms adapted from Lampropoulos et al. [33], Pafka et al. [47]. Such a generator can be used to test that the typing relation guarantees the expected runtime behaviors of programs, e.g. progress and preservation. In addition to the complexity of the coverage property itself (well-typedness), this case study features multiple inductive datatypes (for types, terms, and typing contexts, as shown in Fig. 20), and 13 auxiliary functions. The coverage type of `gen_term_size`, the top-level generator, stipulates that it can generate all terms of a desired type, up to a user-provided size bound:

$$\begin{array}{c}
 \text{gen_term_size} : n : \underbrace{\{v : \text{int} \mid 0 \leq v\}}_{\text{maximum term size}} \rightarrow \underbrace{t : \{v : \text{ty} \mid \top\}}_{\text{type of term}} \rightarrow \underbrace{\Gamma : \{v : \text{tyctx} \mid \top\}}_{\text{typing context}} \rightarrow \\
 \underbrace{[v : \text{term} \mid \text{has_ty}(\Gamma, v, t) \wedge \text{max_app_num}(v, n)]}_{\text{result is well-typed and has at most } n \text{ applications}}
 \end{array}$$

The results of using Poirot to verify that `gen_term_size` meets the above specification are shown in Table 4. The table also reports the results for the most interesting auxiliary functions used by the function. The last column shows that Poirot is able to verify these functions within a reasonable time, ranging from 0.01 to 4.74 seconds. Although more complex functions (as indicated by the column labeled #Branch) require more time to verify, total verification time is nonetheless reasonable: less than 5 seconds in total. Taken together, these results highlight the compositionality of Poirot’s type-based approach: each of the 13 auxiliary functions used

Table 4. Experimental results from the STLC case study. Each function is implemented as a wrapper around a subsidiary function that takes an additional strictly decreasing argument to ensure termination (the original QuickChick implementations uses Coq/Rocq’s Program command for this purpose). These subsidiary functions are responsible for the bulk of the computation, so we report the results for those functions here.

	#Branch	Rec	#LVar	#UP	#Query	(max. # $\forall$, # $\exists$)	total (avg. time) (ms)
type_eq	5	✓	6	4	4	(3, 12)	57.15(14.29)
gen_type	2	✓	9	3	8	(2, 5)	9.47(1.18)
vars_with_type	4	✓	7	5	5	(3, 6)	8.38(1.68)
gen_term_no_app	3	✓	8	12	6	(3, 7)	13.88(2.31)
gen_term_size	4	✓	36	11	32	(5, 16)	4735.37(147.98)

by `gen_term_size` is individually type-checked against its signature; these signatures are then used to verify any procedures that call the function.

Interestingly, the function `type_eq` has a longer average query time than most of other functions, despite having fewer local variables and uninterpreted predicates. This function implements a deterministic equality test, returning `true` when two types are the same and `false` otherwise. Thus, the coverage type of this function degenerates into a singleton type for each of the branches, resulting in stricter queries to the SMT solver that take longer to find a valid witness.

Discussion. To handle the complexity of this benchmark, Poirot requires 16 uninterpreted predicates and 24 axioms, the large majority of which correspond to helper definitions and lemmas from the original development. The predicates that encode typing and the bounds on the number of applications in a term (*has_ty* and *max_num_app*, resp.) come directly from the QuickCheck version, for example. The following axiom encodes the semantic relationship of these predicates

$$\forall \Gamma: \text{tyctx}. \forall t: \text{ty}. \forall e: \text{term}.$$

$$\text{has_ty}(\Gamma, e, t) \iff \exists n: \text{nat}. \text{max_num_app}(e, n) \wedge \text{has_ty}(\Gamma, e, t)$$

and is analogous to the helper lemma `has_ty_max_tau_correct` in the original Coq/Rocq development. In addition, some predicates and axioms are independent of this particular case study: the typing context is implemented as a list of STLC types, and thus we were able to reuse generic predicates and axioms about polymorphic lists.

9.3 Completeness of synthesized generators

Table 5. Quantifying the space of safe and complete test input generators constructed by an automated program synthesis tool.

Benchmark	#Total	#Complete
UniqueList	284	10
SizedList	126	28
SortedList	30	8
SizedTree	103	2
SizedBST	229	54
RedBlackTree	234	2

An underlying hypothesis motivating our work is that writing sound and *complete* test input generators can be subtle and tricky, as demonstrated by our motivating example (Figure 1).

```

1  let rec sized_list_gen
2    (size : int) : (int list) =
3    if (size == 0) then []
4    else
5      if bool_gen () then sized_list_gen (size - 1)
6      else (int_gen ()) :: (sized_list_gen (size - 1))

```

(a) A sound and complete generator.

```

let rec sized_list_gen
  (size : int) : (int list) =
  if (size == 0) then []
  else (int_gen ()) :: (sized_list_gen (size - 1))

```

(b) A sound but incomplete generator.

```

let rec sized_list_gen
  (size : int) : (int list) =
  if (size == 0) then []
  else
    if bool_gen () then sized_list_gen (size - 1)
    else size :: (sized_list_gen (size - 1))

```

(c) Another sound but incomplete generator.

Fig. 21. Three example generators that generate size-bounded lists.

To justify this hypothesis, we repurposed an existing deductive component-based program synthesizer [40] to automatically synthesize correct (albeit possibly incomplete) generators that satisfy a specification given as an overapproximate refinement type; these generators are then fed to Poirot to validate their completeness. We provided the synthesizer with a datatype definition and a set of specifications describing constraints on that datatype the synthesized generator should use, along with a library of functions, including primitive generators such as `nat_gen`, available to the synthesizer for construction. A refinement type-guided enumeration is performed to find all correct programs consistent with the specification. Since the space of these programs is potentially quite large (possibly infinite), we constrain the synthesizer to only generate programs with bounded function call depths; in our experiments, this bound was set to three. The generator outputs all programs that are safe with respect to the specification. Table 5 shows results of this experiment for five of the benchmarks given in Table 3; results for the other benchmarks are similar. We report the total number of synthesized generators (#Total) constructed and the number of those that are *correct and complete* as verified by Poirot (#Complete). The table confirms our hypothesis that the space of complete generators with respect to the supplied coverage type is significantly smaller than the space of safe generators, as defined by an overapproximate refinement type specification.

More concretely, Figure 21 shows three synthesized generators that satisfy the following specification of a list generator that is meant to construct *all* lists no longer than some provided bound:

$$\text{size}:\{v:\text{int} \mid v \leq 0\} \rightarrow [v:\text{int list} \mid \forall u, \text{len}(v, u) \implies (0 \leq u \wedge u \leq \text{size})]$$

Fig. 21b is incomplete because it never generates an empty list when the size parameter `size` is greater than 0. On the other hand, while Fig. 21c does generate empty lists, the else branch of its second conditional has a fixed first element and will therefore never generate lists with distinct elements. The complete generator shown in Fig. 21a incorporates a control-flow path

(line 5) that can non-deterministically choose to make a recursive call to `sized_list_gen` with a smaller size, thereby allowing it to generate lists of variable size up to the `size` bound, including the empty list; another conditional branch uses `int_gen()` to generate a new randomly selected list element, thereby allowing the implementation to generate lists containing distinct elements. We again emphasize that Poirot was able to verify the correct generator and discard the two incorrect generators *automatically*, without any user involvement.

9.4 Case study: checking coverage in practice

Table 6. Experimental results that use monadic combinators from the Gen library of the QCheck framework. The columns are the same as in Table 3; we additionally count the number of type variables (`#a`) and predicate variables (`#P`) for these polymorphic combinators. A number of combinator types do not require uninterpreted predicates since they do not make use of datatypes; `list_repeat` (`list`), `option` (`option`), `pair` (`pair`), and `frequency` (`pair`), do however, employ such predicates.

	#Branch	Rec	#LVar	#a	#P	#UP	#Query	(max. $\#\forall, \#\exists$)	total (avg. time) (ms)
return	1		2	1	0	0	1	(0, 0)	0.14(0.14)
bind	1		5	2	2	0	3	(2, 1)	28.74(9.58)
fmap	1		4	2	2	0	2	(2, 1)	2.23(1.11)
pair	1		5	2	2	2	3	(1, 2)	2.61(0.87)
option	2		7	1	1	3	14	(2, 3)	5.91(0.42)
union	2		4	1	2	0	12	(1, 1)	2.48(0.21)
oneof	1		4	1	1	0	3	(2, 1)	4.35(1.45)
frequency	1		5	1	1	2	3	(3, 2)	3.04(1.01)
list_repeat	2	✓	12	1	1	4	12	(3, 5)	11.48(0.96)
fix	1		5	1	1	0	15	(2, 0)	11.23(0.75)

In this section, we focus on test input generators from real-world applications as well as the underlying combinators provided by mainstream PBT frameworks.

The experimental results of verifying the monadic combinators from Sec. 8 are shown in Table 6. These combinators are relatively simple, and Poirot is able to verify them within 0.03 seconds with small variance.

Table 7. Experimental results for test generators of open-source repositories. The columns follow the same format as in Table 3, with an additional column showing the number of combinators (`#C`) used by these generators. The verification time in the last column is given in seconds.

	#Branch	Rec	#LVar	#UP	#C	#Query	(max. $\#\forall, \#\exists$)	total (avg. time) (s)
Tezos	7	✓	40	15	3	73	(3, 35)	10.75(0.15)
Xen API	1		21	5	3	24	(1, 16)	6.51(0.27)
Vellvm	10	✓	41	19	4	84	(4, 19)	2.05(0.02)
Herdtools7	1		32	10	3	90	(2, 8)	2.08(0.02)
Zipperposition	7	✓	71	16	4	134	(2, 49)	15.81(0.12)

We also apply Poirot to identify coverage completeness violations across five open-source repositories that use the QCheck framework to test their implementation. Each project deals with a different real-world application domain: blockchain [59], virtualization [63], compiler verification [67], weak memory model testing [39], and automated theorem proving [6]. The details of these projects are as follows:

- (1) **Tezos**. Tezos [59] is a blockchain platform implemented in OCaml that supports decentralized assets and applications. Tezos writes its own generators to construct

block trees, i.e., trees of block nodes representing the shape of a blockchain, where blocks are expected to have unique hash codes.

- (2) **Xen API.** Xen API [63] is a management API for controlling the virtualization environments of the Xen Project Hypervisor [64]. We selected the file descriptor generator of the API, which produces random file descriptors with different attributes, e.g., file size, kind, read and write delay time. The generated test inputs are expected to be a subset of valid Unix file descriptors (e.g., not a symbolic link).
- (3) **Vellvm.** Vellvm [67] is a formal verification framework for the LLVM IR, implemented in both OCaml and Coq/Rocq. We selected the generator for LLVM values used to provide as arguments of various LLVM functions under test. Given a specific LLVM type, this generator is expected to generate all well-typed LLVM values.
- (4) **Herdtools7.** Herdtools7 [39] is a collection of tools for modeling and analyzing concurrent programs with weak memory models. The generator *Carpenter* in Herdtools7 produces random ASTs of the Arm Architecture Specification Language [54], where the generator is expected to generate every AST of a specified size.
- (5) **Zipperposition.** Zipperposition [6] is an automated theorem prover that supports polymorphic types and higher-order unification. Test generators are used to produce valid first-order logic (e.g., $\forall x:\alpha.\exists y:\alpha.x \neq y$) and lambda-free higher-order logic formulae.

We used our polymorphic version of Poirot to check that these test generators can cover all test inputs that satisfy the precondition of the program under test. Thus, we type check generators against coverage types that encode specifications extracted from README files, documentation, and comments in the source code. As shown in Table 7, these test generators are complex, including 1-10 control flow branches, 21-71 local variables, and 5-19 uninterpreted predicates. Most uninterpreted predicates are directly converted from datatype constructors (e.g., *cons* for list datatype). Notably, Poirot reports that *none* of these generators are coverage complete with respect to the preconditions we expected, with verification times ranging from 2.08 to 15.81 seconds. Due to the sophisticated nature of realistic test generators, verification takes longer compared to previous benchmarks, though it remains within acceptable limits. One observation is that the result of type checking does not significantly affect the verification time, which differs from standard refinement type checkers that can return early when detecting safety violations in one control flow. This is because Poirot cannot determine the coverage completeness of a program without analyzing *all* corresponding control flows, as discussed in Sec. 4.

In the rest of this section, we highlight three representative categories of coverage incompleteness detected by Poirot for these benchmarks, examine the underlying causes, and suggest candidate fixes to address these coverage gaps.

Limited sampling range. Manually written test input generators often rely on restricted sampling ranges, resulting in coverage completeness violations. For example, Vellvm [67] generates LLVM `int32` and LLVM `int64` values only within the range 0–10000, and the generators in the herdtools7 framework [39] also restrict integer literals appearing in theorems to this interval. These generators typically use QCheck’s built-in `Gen.nat` combinator, which only produces natural numbers below 10000. While developers adopt this restriction under the assumption that smaller values are more likely to detect bugs, such approaches (theoretically)

```

1  type 'a tree =
2  | Leaf of 'a
3  | Node1 of ('a * 'a tree)
4  | Node2 of ('a * 'a tree * 'a tree)
5
6  let rec tree_gen (blocks: block list): block tree gen =
7  match block with
8  | [x] -> return (Leaf x)
9  | x :: xs -> (
10     ... (* the case producing Leaf *)
11     ... (* the case producing Node1 *)
12     int_bound (List.length xs - 1) >=> fun n ->
13     let left, right = List.split_n n xs in
14     tree_gen left >=> fun left ->
15     tree_gen right >=> fun right ->
16     return (Node2 (x, left, right))
17
18  tree_gen (Block.set_to_list unique_nonempty_block_gen)

```

Fig. 22. A tree-like block chain generator.

compromise completeness. One solution is to run two sets of tests, one with smaller values and one with the full range of values. Alternatively, it is straightforward to build a repaired version of this generator by using QCheck’s `frequency` combinator to combine the original with the default, unconstrained generator, adjusting the weights so that it is biased towards the original input space while retaining the possibility of producing other values:

```

let nat = frequency 2 (function
  | 0 -> (10, Gen.nat)
  | _ -> (1, full_nat_gen))

```

Fixed values. Zipperposition [6] uses the `oneof` combinator to restrict variable names to one of three fixed options: X, Y, or Z. Similarly, the Xen hypervisor [63] constrains file sizes to 9 predetermined lengths and only uses one of 4 fixed float number (e.g., 0.001, 0.01, 0.1, and 0.4) as file read and write delay times. While such choices may reflect the developer’s intuition about likely corner cases, they also prevent the exploration of errors outside these considerations. The repair strategies proposed earlier apply here as well.

Datatype transformations. Poirot identified an interesting coverage completeness violation in the Tezos blockchain project [59], where test input generators construct block trees, i.e., trees of block nodes representing the shape of the block chain. Instead of generating such trees directly, Tezos implements a helper function `tree_gen` (shown in Fig. 22) that builds trees from lists of blocks. Specifically, `tree_gen` accepts a non-empty, duplicate-free list of blocks and returns a generator that produces a tree with non-duplicate blocks. The code comments state that this approach allows better control over sampling distributions and uniqueness. Thus, the expected coverage refinement type of `tree_gen` should be:

$$\text{tree_gen} : l : \{v : \text{block list} \mid \text{uniqList}(v)\} \rightarrow$$

$$\mathcal{M} [v : \text{block tree} \mid \text{uniqTree}(v) \wedge \forall u. \text{memTree}(v, u) \iff \text{mem}(l, u)]$$

However, the actual implementation in Tezos fails to satisfy this specification and is rejected by Poirot. The root of every generated tree (line 16) is deterministically selected as the head

of the input list. Moreover, the use of `List.split_n` partitions the input list into sublists that preserve element ordering, leading to the left subtree always containing earlier elements in the list. Since Tezos applies `tree_gen` to sorted lists (line 18), the resulting block trees consistently place the smallest element at the root, with left subtree elements always being smaller than those in the right subtree. The root cause of this error lies in the complexity of maintaining coverage completeness through transformations between inductive datatypes (e.g., from lists to trees).

One possible solution is to randomly permute the input list of blocks. Another patch is to replace the list input with a set:

```
tree_gen : s:{v: block set |  $\tau$ } →
    M [v: block tree |  $\text{uniqTree}(v) \wedge \forall u. \text{memTree}(v, u) \iff \text{memSet}(s, u)$ ]
```

A coverage complete implementation would then randomly select the root node from this set and partition the set into two disjoint subsets using a randomized split instead of relying on list indexing. We have submitted this patch to the Tezos codebase.

Discussion. As with any verification methodology, the question of whether the completeness violations discovered by Poirot in our case studies are true bugs or represent deliberate trade-offs is one that must ultimately be answered by the authors of these generators. We emphasize, however, that coverage types do enable developers to rigorously specify the intended behaviors of an input generator and to validate that an implementation exhibits those behaviors. The ability to do so has concrete benefits for both the developers of input generators and the test engineers that rely on them.

First, potential incompleteness bugs can help to surface semantic mismatches between the test engineer’s and system developer’s understandings of the precondition of the system under test. In the case of the Tezos case study, for example, the test engineer may have believed that a tree of distinct blocks is isomorphic to a flattened, sorted list of blocks. By identifying this mismatch, Poirot can provide actionable feedback to both parties, enabling them to align their mental models of the system under test.

On the other hand, the coverage type of a generator also provides useful documentation to its clients. As our case studies show, developers may deliberately prioritize efficient sampling over coverage completeness in practice, embedding *ad hoc* restrictions and choices in their implementations. When these trade-offs are not explicitly documented, as is often the case, the users of such generators may inadvertently fail to probe system behaviors on relevant portions of the input space. A generator for a parser or string manipulating program that restricts variable names to `a`, `b`, and `c` will fail to discover errors that can only be triggered by programs with more than three distinct variables. Similarly, only considering small integers may fail to probe arithmetic overflow cases like $-2^{31}/-1$ (the range of `int32` is $-2^{31} \sim 2^{31}-1$). By codifying these sorts of trade-offs as explicit type-level specifications, Poirot helps clients understand and justify the use of a particular generator when testing a system.

Finally, as [Sec. 9.3](#) demonstrated, coverage types can provide useful information when constructing new generators, particularly when doing so automatically [31]. These types can also be helpful when manually defining generators *compositionally* via monadic combinators, e.g., when a test engineer wants to combine generators that sample from different distributions. For example, the Xen API [63] includes the file descriptor generator shown in [Fig. 23](#), which applies the `oneof` combinator to generators for singleton lists and general file descriptor lists.

Currently, these generators are informally described in the accompanying code comment; coverage types provide a rigorous mechanism for capturing these sorts of behaviors along with a principled characterization of how they fit together.

```

1  (* generates 2 kinds of lists:
2    - lists that contain only a single file kind
3    - lists that contain multiple file kinds
4
5    This is important for testing [select], because a single
6    [Unix.S_REG] would cause it to return immediately,
7    making it unlikely that we're actually testing
8    the behaviour for other file descriptors.
9  *)
10 oneof [g1; g2]
```

Fig. 23. A file descriptor generator that uses oneof to combine two generators with different coverage guarantees.

10 Related work

Property-based testing (PBT) is an automated testing approach that uses random tests to determine if a user-defined property is satisfied by the component under test. In a PBT setting, test inputs are expected to satisfy the preconditions of the functions under test and are equipped with a “shrink” procedure that automatically reduces failing inputs to minimal examples for reporting to users. First introduced for Haskell [5] to test the behavioral correctness of simple programs over algebraic datatypes (ADTs), PBT has since been successfully applied to a number of application domains, including operating systems [63], compilers [18], networks [38], memory models [39], web applications [44], and quantum computation [26]. Meanwhile, the properties in PBT have also been extended to richer logics. For instance, O’Connor and Wickström [44] allows users to provide specifications in linear temporal logic (LTL) to guide the testing of interactive applications. Property-based testing can also validate performance specifications, in addition to safety properties. For example, Löscher et al. [38] uses PropEr [52], a PBT framework in Erlang, to verify that the energy consumption of sensor networks remains within a specific bound.

Although PBT test generators can be derived automatically for arbitrary datatypes, handwritten test generators remain widely used. Etna [58] evaluates the test generators of mainstream PBT frameworks in Coq/Rocq and Haskell, concluding that “bespoke generators” (user-defined test generators) perform well in most cases. However, a recent empirical study [21] analyzed the use of property-based testing (PBT) in industrial practice. One key finding is that writing test generators for complex precondition properties (e.g., valid red-black trees) remains a significant challenge, even when PBT frameworks provide extensive combinators and monadic APIs. Developers described the process as “tedious” and “high-effort”, particularly when balancing coverage and input quality (e.g., generating inputs closer to realistic distributions). Poirot’s integration of PBT abstractions within an expressive refinement-type system tailored for coverage provides an automated validation mechanism to ensure generators satisfy rich safety and completeness constraints.

There has also been recent work that reduces user effort in writing high-quality test generators. Tyche [23] enables better visualization of test generator quality, including valid

and invalid test cases, duplicate test cases, and corresponding distributions. Elazar Mittelman et al. [14] demonstrates that test generators for complex ADTs can be improved through combinatorial testing [43], where the domain of input ADT instances can be partitioned by combinations of datatype constructors. Several works have considered how to automatically synthesize sound and complete generators for a target precondition: the QuickChick PBT framework for Coq/Rocq, for example, is capable of deriving generators from preconditions expressed as inductive relations [35, 48]. LaFontaine et al. [31] present a technique for automatically repairing incomplete generators used coverage-type-guided program synthesis; when applied to a control-flow sketch, this technique is capable of synthesizing a coverage-complete generator for a given precondition. The recently proposed Palamedes [22] tool uses deductive synthesis to generate sound and complete generators from precondition that are expressed in Lean using an extensible set of recursion schemes. Another set of works consider the more specific problem of how to automatically adjust the distribution of an input generator. The Dragen tool [41, 42], for example, refines the default input generators derived by the QuickCheck framework by tuning input weights of frequency combinators using additional structural information provided by users. Loaded Dice [60] relaxes this restriction, allowing users to automatically tune the distribution of an arbitrary generator by first lowering it into a probabilistic DSL embedded in Julia and then optimizing those weights based on a user-provided objective function.

A general limitation of PBT is that its effectiveness suffers when the property of interest has a strict precondition [32], because most of the inputs produced by a purely random test generation strategy will be simply discarded. As a result, there has been much recent interest on improving the coverage of test generators with respect to a particular precondition. Proposed solutions range from adopting ideas from fuzzing [10, 66] to intelligently mutate the outputs produced by the generator [34, 46], to focusing on generators for particular classes of inputs (e.g., well-typed programs) [16, 47, 65], to automatically building complete-by-construction generators [4, 33, 35]. While sharing broadly similar goals with these proposals, our approach differs significantly in its framing of coverage in purely type-theoretic terms. This fundamental change in perspective allows us to statically and compositionally verify coverage properties of a generator without the need for any form of instrumentation on, or runtime monitoring of, the program under test (as in [10, 34]). Unlike other approaches that have also considered the verification of a generator’s coverage properties [12, 13, 49] using a mechanized proof assistant, our proposed type-based framing is highly-automated and inherently compositional. Expressing coverage as part of a type system also allows us to be agnostic to (a) how generators are constructed, (b) the particulars of the application domain [16, 18, 47, 65], and (c) the specific structure of the properties being tested [33, 35]. Poirot’s ability to specify and type-check a complex coverage property depends only on whether we can express a desired specification using available uninterpreted predicates.

A number of logics have been proposed for reasoning about underapproximations of program behavior, including the recently developed incorrectness logic (IL) [45, 53], reverse Hoare logic (RHL) [9], and dynamic logic (DL) [51]. Both IL and RHL are formalisms similar to Hoare logic, but support composable specifications that assert underapproximate postconditions, with IL adding special post-assertions for error states. IL was originally proposed as a way of formalizing the conditions under which a particular program point (say an error state) is guaranteed to be reachable, and has recently been used in program analyses

that discover memory errors [37]. DL, in contrast, reinterprets Hoare logic as a multi-modal logic equipped with operators for reasoning about the existence of executions that end in a state satisfying some desired postcondition. This paper instead provides the first development that interprets these notions in the context of a type system for a rich functional language. While our ideas are formulated in the context of verifying coverage properties for test input generators, we believe our framework can be equally adept in expressing type-based program analyses for bug finding or compiler optimizations.

Our focus on reasoning about coverage properties of test input generators distinguishes our approach, in obvious ways from other refinement type-based testing solutions such as TARGET [57]. Nonetheless, our setup follows the same general verification playbook as Liquid Types [29, 62] — our underapproximate specifications are identical to their overapproximate counterpart, except that we syntactically distinguish the return types for functions to reflect their expected underapproximate (rather than overapproximate) behavior. An important consequence of this design is that the burden of specifying and checking the coverage behavior of a program is no greater than specifying its safety properties.

Another related line of work has explored how to reason about the distribution of data produced by a function [1, 2], with a focus on ensuring that these distributions are free of unwanted biases. These works have considered decision-making and machine-learning applications, in which these sorts of fairness properties can be naturally encoded as (probabilistic) formulas in real arithmetic. In contrast, coverage types can only verify that a generator has a nonzero probability of producing a particular output. Extending our type system and its guarantees to provide stronger fairness guarantees about the distribution of the sorts of discrete data produced by test input generators is an exciting direction for future work.

11 Conclusion

This paper adapts principles of underapproximate reasoning found in recent work on Incorrectness Logic to the specification and automated verification of test input generators used in modern property-based testing systems. Specifications are expressed in the language of refinement types, augmented with coverage types, types that reflect underapproximate constraints on program behavior. A novel bidirectional type-checking algorithm enables an expressive form of inference over these types. Our experimental results demonstrate that our approach is capable of verifying both sophisticated hand-written generators, as well as being able to successfully identify type-correct (in an overapproximate sense) but coverage-incomplete generators produced from a deductive refinement type-aware synthesizer. We also demonstrate how polymorphic extensions to our type system can be used to successfully detect coverage violations of complex test generators implemented using monadic combinators in real-world developments.

Acknowledgments

This work was supported in part by DARPA under the grants HR0011-19-C-0073 and HR0011-22-90062, the Department of Defense under grant FA8649-21-P-1511, and the NSF under grant CCF-2321680.

Data availability

An artifact containing this implementation, our benchmark suite, results, and the corresponding Rocq proofs (i.e., our soundness theorem, Theorem 5.3) is publicly available on Zenodo [70]. The benchmark suite of our new case study is available on GitHub at <https://github.com/zhezhouzz/CoverageType/tree/jfp> (commit c6fa15d).

References

- [1] Aws Albarghouthi, Loris D’Antoni, Samuel Drews, and Aditya V. Nori. 2017. FairSquare: Probabilistic Verification of Program Fairness. *Proc. ACM Program. Lang.* 1, OOPSLA, Article 80 (oct 2017), 30 pages. doi:10.1145/3133904
- [2] Osbert Bastani, Xin Zhang, and Armando Solar-Lezama. 2019. Probabilistic Verification of Fairness Properties via Concentration. *Proc. ACM Program. Lang.* 3, OOPSLA, Article 118 (Oct 2019), 27 pages. doi:10.1145/3360544
- [3] Koen Claessen. 2006. *QuickCheck*. <https://hackage.haskell.org/package/QuickCheck>
- [4] Koen Claessen, Jonas Duregård, and Michał H. Pałka. 2014. Generating Constrained Random Data with Uniform Distribution. In *Functional and Logic Programming*, Michael Codish and Eijiro Sumii (Eds.). Springer International Publishing, Cham, 18–34. doi:10.1007/978-3-319-07151-0_2
- [5] Koen Claessen and John Hughes. 2000. QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs. In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP ’00)*. Association for Computing Machinery, New York, NY, USA, 268–279. doi:10.1145/351240.351266
- [6] Simon Cruanes. 2012. *zipperposition*. <https://github.com/sneeuwballen/zipperposition>
- [7] Simon Cruanes. 2013. *QCheck*. <https://github.com/c-cube/qcheck>
- [8] Leonardo de Moura and Nikolaj Bjørner. 2008. Z3: An Efficient SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems*, C. R. Ramakrishnan and Jakob Rehof (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 337–340. doi:10.1007/978-3-540-78800-3_24
- [9] Edsko de Vries and Vasileios Koutavas. 2011. Reverse Hoare Logic. In *Software Engineering and Formal Methods*, Gilles Barthe, Alberto Pardo, and Gerardo Schneider (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 155–171. doi:10.1007/978-3-642-24690-6_12
- [10] Stephen Dolan. 2022. *Crowbar*. <https://github.com/stedolan/crowbar>
- [11] Jana Dunfield and Neel Krishnaswami. 2021. Bidirectional Typing. *ACM Comput. Surv.* 54, 5, Article 98 (may 2021), 38 pages. doi:10.1145/3450952
- [12] Peter Dybjer, Qiao Haiyan, and Makoto Takeyama. 2003. Combining Testing and Proving in Dependent Type Theory. In *Theorem Proving in Higher Order Logics, 16th International Conference, TPHOLs 2003, Rom, Italy, September 8-12, 2003, Proceedings (Lecture Notes in Computer Science, Vol. 2758)*, David A. Basin and Burkhard Wolff (Eds.). Springer, 188–203. doi:10.1007/10930755_12
- [13] Peter Dybjer, Qiao Haiyan, and Makoto Takeyama. 2004. Random Generators for Dependent Types. In *Theoretical Aspects of Computing - ICTAC 2004, First International Colloquium, Guiyang, China, September 20-24, 2004, Revised Selected Papers (Lecture Notes in Computer Science, Vol. 3407)*, Zhiming Liu and Keijiro Araki (Eds.). Springer, 341–355. doi:10.1007/978-3-540-31862-0_25
- [14] Segev Elazar Mittelman, Aviel Resnick, Ivan Perez, Alwyn E. Goodloe, and Leonidas Lampropoulos. 2023. Don’t Go Down the Rabbit Hole: Reprioritizing Enumeration for Property-Based Testing. In *Proceedings of the 16th ACM SIGPLAN International Haskell Symposium* (Seattle, WA, USA) (*Haskell 2023*). Association for Computing Machinery, New York, NY, USA, 59–71. doi:10.1145/3609026.3609730
- [15] FastCheck 2022. *fast-check: Property based testing for JavaScript and TypeScript*. <https://dubzzz.github.io/fast-check.github.com/>
- [16] Burke Fetscher, Koen Claessen, Michał H. Pałka, John Hughes, and Robert Bruce Findler. 2015. Making Random Judgments: Automatically Generating Well-Typed Terms from the Definition of a Type-System. In *Programming Languages and Systems - 24th European Symposium on Programming, ESOP 2015, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2015, London, UK, April 11-18, 2015. Proceedings (Lecture Notes in Computer Science, Vol. 9032)*, Jan Vitek (Ed.). Springer, 383–405. doi:10.1007/978-3-662-46669-8_16

- [17] Cormac Flanagan, Amr Sabry, Bruce F. Duba, and Matthias Felleisen. 1993. The Essence of Compiling with Continuations. In *Proceedings of the ACM SIGPLAN 1993 Conference on Programming Language Design and Implementation* (Albuquerque, New Mexico, USA) (PLDI '93). Association for Computing Machinery, New York, NY, USA, 237–247. doi:10.1145/155090.155113
- [18] Justin Frank, Benjamin Quiring, and Leonidas Lampropoulos. 2024. Generating Well-Typed Terms That Are Not “Useless”. *Proc. ACM Program. Lang.* 8, POPL, Article 77 (Jan. 2024), 22 pages. doi:10.1145/3632919
- [19] Kimball Germane and Jay McCarthy. 2021. Newly-single and Loving It: Improving Higher-Order Must-Alias Analysis with Heap Fragments. *Proc. ACM Program. Lang.* 5, ICFP (2021), 1–28. doi:10.1145/3473601
- [20] Patrice Godefroid, Aditya V. Nori, Sriram K. Rajamani, and Sai Deep Tetali. 2010. Compositional May-Must Program Analysis: Unleashing the Power of Alternation. In *Proceedings of the 37th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Madrid, Spain) (POPL '10). Association for Computing Machinery, New York, NY, USA, 43–56. doi:10.1145/1706299.1706307
- [21] Harrison Goldstein, Joseph W. Cutler, Daniel Dickstein, Benjamin C. Pierce, and Andrew Head. 2024. Property-Based Testing in Practice. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering* (Lisbon, Portugal) (ICSE '24). Association for Computing Machinery, New York, NY, USA, Article 187, 13 pages. doi:10.1145/3597503.3639581
- [22] Harrison Goldstein, Hila Peleg, Cassia Torczon, Daniel Sainati, Leonidas Lampropoulos, and Benjamin C. Pierce. 2025. The Search for Constrained Random Generators. arXiv:2511.12253 [cs.PL] <https://arxiv.org/abs/2511.12253>
- [23] Harrison Goldstein, Jeffrey Tao, Zac Hatfield-Dodds, Benjamin C. Pierce, and Andrew Head. 2024. Tyche: Making Sense of PBT Effectiveness. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology* (Pittsburgh, PA, USA) (UIST '24). Association for Computing Machinery, New York, NY, USA, Article 10, 16 pages. doi:10.1145/3654777.3676407
- [24] John Hatcliff and Olivier Danvy. 1994. A Generic Account of Continuation-Passing Styles. In *Proceedings of the 21st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Portland, Oregon, USA) (POPL '94). Association for Computing Machinery, New York, NY, USA, 458–471. doi:10.1145/174675.178053
- [25] C. A. R. Hoare. 1969. An Axiomatic Basis for Computer Programming. *Commun. ACM* 12, 10 (oct 1969), 576–580. doi:10.1145/363235.363259
- [26] Shahin Honarvar, Mohammad Reza Mousavi, and Rajagopal Nagarajan. 2020. Property-based Testing of Quantum Programs in Q#. In *Proceedings of the IEEE/ACM 42nd International Conference on Software Engineering Workshops* (Seoul, Republic of Korea) (ICSEW'20). Association for Computing Machinery, New York, NY, USA, 430–435. doi:10.1145/3387940.3391459
- [27] Hypothesis 2022. *Hypothesis*. <https://github.com/HypothesisWorks/hypothesis/tree/master/hypothesis-python>
- [28] Suresh Jagannathan, Peter Thiemann, Stephen Weeks, and Andrew K. Wright. 1998. Single and Loving It: Must-Alias Analysis for Higher-Order Languages. In *POPL '98, Proceedings of the 25th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, San Diego, CA, USA, January 19-21, 1998*, David B. MacQueen and Luca Cardelli (Eds.). ACM, 329–341. doi:10.1145/268946.268973
- [29] Ranjit Jhala and Niki Vazou. 2021. Refinement Types: A Tutorial. *Found. Trends Program. Lang.* 6, 3-4 (2021), 159–317. doi:10.1561/25000000032
- [30] Gowtham Kaki and Suresh Jagannathan. 2014. A relational framework for higher-order shape analysis. In *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming* (Gothenburg, Sweden) (ICFP '14). Association for Computing Machinery, New York, NY, USA, 311–324. doi:10.1145/2628136.2628159
- [31] Patrick LaFontaine, Zhe Zhou, Ashish Mishra, Suresh Jagannathan, and Benjamin Delaware. 2025. We’ve Got You Covered: Type-Guided Repair of Incomplete Input Generators. *Proc. ACM Program. Lang.* 9, OOPSLA2, Article 380 (Oct. 2025), 29 pages.
- [32] Leonidas Lampropoulos. 2018. *Random Testing for Language Design*. Ph.D. Dissertation. University of Pennsylvania. <https://repository.upenn.edu/edissertations/2879>
- [33] Leonidas Lampropoulos, Diane Gallois-Wong, Cătălin Hrițcu, John Hughes, Benjamin C. Pierce, and Li-yao Xia. 2017. Beginner’s Luck: A Language for Property-Based Generators. *SIGPLAN Not.* 52, 1 (jan 2017), 114–129. doi:10.1145/3093333.3009868
- [34] Leonidas Lampropoulos, Michael Hicks, and Benjamin C. Pierce. 2019. Coverage Guided, Property Based Testing. *Proc. ACM Program. Lang.* 3, OOPSLA, Article 181 (oct 2019), 29 pages. doi:10.1145/3360607

- [35] Leonidas Lampropoulos, Zoe Paraskevopoulou, and Benjamin C. Pierce. 2018. Generating Good Generators for Inductive Relations. *Proc. ACM Program. Lang.* 2, POPL (2018), 45:1–45:30. doi:10.1145/3158133
- [36] Leonidas Lampropoulos and Benjamin C. Pierce. 2022. *QuickChick: Property-Based Testing in Coq*. Software Foundations, Vol. 4. Electronic textbook. Version 1.3.1, <https://softwarefoundations.cis.upenn.edu>.
- [37] Quang Loc Le, Azalea Raad, Jules Villard, Josh Berdine, Derek Dreyer, and Peter W. O'Hearn. 2022. Finding Real Bugs in Big Programs with Incorrectness Logic. *Proc. ACM Program. Lang.* 6, OOPSLA (2022), 1–27. doi:10.1145/3527325
- [38] Andreas Löschner, Konstantinos Sagonas, and Thiemo Voigt. 2015. Property-based testing of sensor networks. In *2015 12th Annual IEEE International Conference on Sensing, Communication, and Networking (SECON)*. 100–108. doi:10.1109/SAHCN.2015.7338296
- [39] Luc Maranget. 2015. *herdtools7*. <https://github.com/herd/herdtools7>
- [40] Ashish Mishra and Suresh Jagannathan. 2022. Specification-Guided Component-Based Synthesis from Effectful Libraries. *Proc. ACM Program. Lang.* 6, OOPSLA2, Article 147 (oct 2022), 30 pages. doi:10.1145/3563310
- [41] Agustín Mista and Alejandro Russo. 2019. Generating random structurally rich algebraic data type values. In *Proceedings of the 14th International Workshop on Automation of Software Test (Montreal, Quebec, Canada) (AST '19)*. IEEE Press, 48–54. doi:10.1109/AST.2019.00013
- [42] Agustín Mista, Alejandro Russo, and John Hughes. 2018. Branching processes for QuickCheck generators. In *Proceedings of the 11th ACM SIGPLAN International Symposium on Haskell (St. Louis, MO, USA) (Haskell 2018)*. Association for Computing Machinery, New York, NY, USA, 1–13. doi:10.1145/3242744.3242747
- [43] Changhai Nie and Hareton Leung. 2011. A survey of combinatorial testing. *ACM Comput. Surv.* 43, 2, Article 11 (Feb. 2011), 29 pages. doi:10.1145/1883612.1883618
- [44] Liam O'Connor and Oskar Wickström. 2022. Quickstrom: property-based acceptance testing with LTL specifications. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation (San Diego, CA, USA) (PLDI 2022)*. Association for Computing Machinery, New York, NY, USA, 1025–1038. doi:10.1145/3519939.3523728
- [45] Peter W. O'Hearn. 2019. Incorrectness Logic. *Proc. ACM Program. Lang.* 4, POPL, Article 10 (dec 2019), 32 pages. doi:10.1145/3371078
- [46] Rohan Padhye, Caroline Lemieux, Koushik Sen, Mike Papadakis, and Yves Le Traon. 2019. Semantic Fuzzing with Zest. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*. Association for Computing Machinery, New York, NY, USA, 329–340. doi:10.1145/3293882.3330576
- [47] Michał H. Pałka, Koen Claessen, Alejandro Russo, and John Hughes. 2011. Testing an Optimising Compiler by Generating Random Lambda Terms. In *Proceedings of the 6th International Workshop on Automation of Software Test (Waikiki, Honolulu, HI, USA) (AST '11)*. Association for Computing Machinery, New York, NY, USA, 91–97. doi:10.1145/1982595.1982615
- [48] Zoe Paraskevopoulou, Aaron Eline, and Leonidas Lampropoulos. 2022. Computing correctly with inductive relations. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation (San Diego, CA, USA) (PLDI 2022)*. Association for Computing Machinery, New York, NY, USA, 966–980. doi:10.1145/3519939.3523707
- [49] Zoe Paraskevopoulou, Cătălin Hrițcu, Maxime Dénès, Leonidas Lampropoulos, and Benjamin C. Pierce. 2015. Foundational Property-Based Testing. In *Interactive Theorem Proving*, Christian Urban and Xingyuan Zhang (Eds.). Springer International Publishing, Cham, 325–343. doi:10.1007/978-3-319-22102-1_22
- [50] Benjamin C. Pierce. 2002. *Types and Programming Languages* (1st ed.). The MIT Press.
- [51] Vaughan R Pratt. 1976. Semantical consideration on Floyd-Hoare logic. In *17th Annual Symposium on Foundations of Computer Science (sfcs 1976)*. IEEE, 109–121.
- [52] propEr 2018. *propEr*. <https://proper-testing.github.io/publications.html>
- [53] Azalea Raad, Josh Berdine, Hoang-Hai Dang, Derek Dreyer, Peter O'Hearn, and Jules Villard. 2020. Local Reasoning About the Presence of Bugs: Incorrectness Separation Logic. In *Computer Aided Verification: 32nd International Conference, CAV 2020, Los Angeles, CA, USA, July 21–24, 2020, Proceedings, Part II* (Los Angeles, CA, USA). Springer-Verlag, Berlin, Heidelberg, 225–252. doi:10.1007/978-3-030-53291-8_14
- [54] Alastair David Reid. 2019. *Defining interfaces between hardware and software: Quality and performance*. Ph. D. Dissertation. University of Glasgow.
- [55] RustCheck 2021. *Crate for PBT in Rust*. <https://github.com/BurntSushi/quickcheck>
- [56] ScalaCheck 2021. *ScalaCheck*. <https://scalacheck.org/>

- [57] Eric L. Seidel, Niki Vazou, and Ranjit Jhala. 2015. Type Targeted Testing. In *Programming Languages and Systems*. Springer Berlin Heidelberg, Berlin, Heidelberg, 812–836. doi:10.1007/978-3-662-46669-8_33
- [58] Jessica Shi, Alperen Keles, Harrison Goldstein, Benjamin C. Pierce, and Leonidas Lampropoulos. 2023. Etna: An Evaluation Platform for Property-Based Testing (Experience Report). *Proc. ACM Program. Lang.* 7, ICFP, Article 218 (Aug. 2023), 17 pages. doi:10.1145/3607860
- [59] Tezos. 2019. *Tezos*. <https://gitlab.com/tezos/tezos>
- [60] Ryan Tjoa, Poorva Garg, Harrison Goldstein, Todd Millstein, Benjamin C. Pierce, and Guy Van den Broeck. 2025. Tuning Random Generators: Property-Based Testing as Probabilistic Programming. *Proc. ACM Program. Lang.* 9, OOPSLA2, Article 304 (Oct. 2025), 28 pages. doi:10.1145/3763082
- [61] Niki Vazou, Patrick M. Rondon, and Ranjit Jhala. 2013. Abstract refinement types. In *Proceedings of the 22nd European Conference on Programming Languages and Systems (Rome, Italy) (ESOP'13)*. Springer-Verlag, Berlin, Heidelberg, 209–228. doi:10.1007/978-3-642-37036-6_13
- [62] Niki Vazou, Eric L. Seidel, Ranjit Jhala, Dimitrios Vytiniotis, and Simon Peyton-Jones. 2014. Refinement Types for Haskell. *SIGPLAN Not.* 49, 9 (aug 2014), 269–282. doi:10.1145/2692915.2628161
- [63] Xen-Project. 2010. *Xen API*. <https://github.com/xapi-project/xen-api>
- [64] Xen-Project. 2010. *Xen Project*. <https://xenproject.org/>
- [65] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and Understanding Bugs in C Compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation (San Jose, California, USA) (PLDI '11)*. Association for Computing Machinery, New York, NY, USA, 283–294. doi:10.1145/1993498.1993532
- [66] Michal Zalewski. 2020. *American Fuzzy Lop*. <https://github.com/google/aflop>
- [67] Steve Zdancewic, Yannick Zakowsky, Calvin Beck, Irene Yoon, and Hanxi Chen. 2012. *Vellvm*. <https://github.com/vellvm/vellvm>
- [68] Zhe Zhou, Robert Dickerson, Benjamin Delaware, and Suresh Jagannathan. 2021. Data-driven abductive inference of library specifications. *Proc. ACM Program. Lang.* 5, OOPSLA (2021), 1–29. doi:10.1145/3485493
- [69] Zhe Zhou, Ashish Mishra, Benjamin Delaware, and Suresh Jagannathan. 2023. Covering All the Bases: Type-Based Verification of Test Input Generators. *Proc. ACM Program. Lang.* 7, PLDI, Article 157 (June 2023), 24 pages. doi:10.1145/3591271
- [70] Zhe Zhou, Ashish Mishra, Benjamin Delaware, and Suresh Jagannathan. 2023. PLDI2023 Artifact: Covering All the Bases: Type-Based Verification of Test Input Generators. doi:10.5281/zenodo.7811004
- [71] Zhe Zhou, Ashish Mishra, Benjamin Delaware, and Suresh Jagannathan. 2025. Polymorphic Coverage Types. arXiv:2304.03393 [cs.PL] <https://arxiv.org/abs/2304.03393v3> <https://arxiv.org/abs/2304.03393v3>